

PROPEASE

Main Project report submitted to
Kristu Jyoti College of Management and Technology,
Changanacherry

In partial fulfillment of the requirement for the award of the
BACHELOR OF COMPUTER APPLICATION
BY

ANANTHAPADMANABHAN M

220021085134

Under the guidance of
Dr. SUSHEEL GEORGE JOSEPH



POST GRADUATE DEPARTMENT OF COMPUTER APPLICATION
KRISTU JYOTI COLLEGE OF MANAGEMENT AND TECHNOLOGY,
CHANGANACHERRY, MARCH 2025

**KRISTU JYOTI COLLEGE OF MANAGEMENT AND TECHNOLOGY,
DEPARTMENT OF COMPUTER APPLICATION**



CERTIFICATE

Certify that the report entitled “**PROPEASE**” is a bonified record of the main project work done by **ANANTHAPADMANABHAN M (220021085134)** under our guidance and supervision is submitted in partial fulfillment of the Bachelor Degree in Computer Applications, awarded by Mahatma Gandhi University Kerala and that no part of this work has been submitted earlier for the award of any other degree.

Dr. Susheel George Joseph

Project Guide

Mr. Roji Thomas

HOD

Department Seal

Ms. Aby Rose Varghese

Project Coordinator

Rev. Dr. Joshy George

Principal

Submitted for presentation and viva held on

Examiner(s)

1.

2.



CERTIFICATE

This is to certify that **Mr. ARJUN SANTHOSH. (Reg No: 220021085141)** Bachelor of Computer Applications VI semester student of **KRISTU JYOTI COLLEGE OF MANAGEMENT AND TECHNOLOGY** affiliated to Mahatma Gandhi university, has done project work entitled **“SPOTIFY LITE”** in PYTHON+FLUTTER under the guidance of our senior faculties towards the fulfillment of the award of “Bachelor of Computer Applications” during the period of January 2025 to April 2025.

He successfully completed the project and during the period he was methodical and hardworking.

For RISS TECHNOLOGIES

Chief Executive Officer



DECLARATION

I hereby declare that the project work entitled “**PROPEASE**” submitted to Mahatma Gandhi University in partial fulfillment of requirement for the award of degree of Bachelor of Computer Application from Kristu Jyoti College of Management and Technology, Changanassery is a record of bonafide work done under the guidance of **Dr. Susheel George Joseh, Project guide, Kristu Jyoti College of Management and Technology, Changanassery.** This project has not been submitted in partial or fulfillment of any other degree/diploma/fellowship or similar of the university or any other university.

Place: Changanassery

ANANTHAPADMANABHAN M

Date:

220021085141

ACKNOWLEDGEMENT

I would like to express my thankfulness towards a number of people who have been instrument in making this project. First of all, I thank the college management who has been with me and provide most helpful facilities throughout the completion of the project.

I am greatly indebted to **Rev. Dr. Joshy George, Principal, Kristu Jyoti College**, who wholeheartedly gave me the permission and whose advice was a real encouragement.

I record my sincere thanks and gratitude to **Mr. Roji Thomas, HOD, Department of Computer Applications**, for his valuable advice and encouragement during the course of this project work.

I am extremely grateful to **Dr. Susheel George Joseph, Project guide** and **Ms. Aby Rose Varghese, Project Coordinator**, for the valuable guidance and suggestions provided which helped me to overcome the major problems encountered during the completion of this project.

ARJUN SANTHOSH

220021085141

ABSTRACT

Project Abstract: Real Estate Platform (MERN Stack)

The Real Estate Platform is a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) to offer an intuitive and secure platform for users to list, buy, and rent properties. Designed to cater to both landlords and tenants, the platform provides a seamless user experience with real-time communication, robust search capabilities, and scalable architecture. The app is optimized for modern web usage, ensuring performance, security, and user-centric functionality.

Key Features:

- 1. User Authentication:** The platform incorporates a secure login and registration system powered by JWT (JSON Web Tokens). Users can easily register, log in, and access their personalized dashboards, ensuring secure access and data privacy.
- 2. Property Listings:** Users (landlords) can create, update, and delete property listings. Each listing includes detailed information such as property type, price, location, size, and amenities, allowing tenants to browse and find their ideal properties.
- 3. Search & Filters:** With advanced search functionality, users can find properties based on various filters, such as location, price range, property size, and amenities. This makes the platform highly efficient for users looking for specific criteria in properties.
- 4. Saved Listings:** Users can save properties they are interested in for future reference. This feature allows tenants to revisit properties and landlords to track potential buyers or tenants.
- 5. Chat System:** The built-in real-time chat feature allows seamless communication between landlords and potential buyers or tenants. This enables users to inquire about listings, negotiate prices, and discuss further details without leaving the platform.

6. **Geolocation Support:** The platform stores property locations using latitude and longitude, which are displayed on interactive maps, enhancing the browsing experience. This ensures users can see the exact location of properties in relation to nearby amenities and services.
7. **Scalability & Security:** Built with MongoDB for flexible data storage, Prisma ORM for efficient database management, and Express.js for API development, the platform is designed to scale as the user base and data volume grow. Security is a top priority, ensuring safe interactions and transactions for all users.

Technology Stack:

- **MongoDB:** Used for storing and managing user and property data in a NoSQL format, providing flexibility and scalability.
- **Prisma ORM:** Simplifies database queries and management, ensuring efficient communication with MongoDB.
- **Express.js:** A robust backend framework for building RESTful APIs, handling HTTP requests, and ensuring seamless integration with the frontend.
- **React.js:** A popular JavaScript framework for building responsive, dynamic, and interactive user interfaces.
- **Node.js:** JavaScript runtime environment that enables server-side development, handling multiple client requests efficiently.
- **JWT Authentication:** Implements secure authentication mechanisms for safe login and access control.
- **Cloudinary:** Used for hosting and managing property images, ensuring fast and reliable image loading.
- **Tailwind CSS:** A utility-first CSS framework that aids in creating modern, responsive, and aesthetically pleasing designs.

Project Structure:

- **/api:**
 - **models/:** Contains MongoDB data models for users, properties, and other entities.

- **routes/:** Defines the RESTful API routes for property listing, user authentication, and chat management.
- **controllers/:** Contains business logic for handling requests from the frontend.
- **middleware/:** Includes authentication middleware, error handling, and validation logic.
- **config/:** Stores configuration files, including environment variables and database connections.
- **server.js:** Main entry point for the backend server, handling API requests and middleware.
- **/client:**
 - **src/:** Contains the source code for the React frontend application.
 - **components/:** Houses reusable UI components like buttons, cards, forms, etc.
 - **pages/:** Contains the page components, representing various views such as home, profile, listings, etc.
 - **hooks/:** Custom React hooks for state management and API interactions.
 - **App.js:** The main React application component that sets up the app's routing and layout.
 - **index.js:** The entry point for the React application, rendering the app to the DOM.

Conclusion:

This Real Estate Platform offers a comprehensive and user-friendly solution for property transactions. By leveraging the MERN stack, the platform provides scalability, real-time communication, secure authentication, and a seamless browsing experience. The application is designed to meet the needs of both landlords and tenants, ensuring a smooth, intuitive, and engaging experience while facilitating property transactions in a modern digital environment.

INDEX

Serial No.	Description	Page No.
1.	CHAPTER 1: INTRODUCTION 1.1: INTRODUCTION ABOUT THE PROJECT 1.2: PROBLEM STATEMENT 1.3: SCOPE AND RELEVANCE OF THE PROJECT 1.4: OBJECTIVES	
2.	CHAPTER 2: SYSTEM ANALYSIS 2.1: INTRODUCTION 2.2: EXISTING SYSTEM 2.3: PROPOSED SYSTEM 2.4: FEASIBILITY STUDY 2.5: SOFTWARE ENGINEERING PARADIGM APPLIED	
3.	CHAPTER 3: SYSTEM DESIGN 3.1: INTRODUCTION 3.2: DATABASE DESIGN 3.3: PROCESS DESIGN DATAFLOW DIAGRAM 3.4: OBJECT ORIENTED DESIGN UML DIAGRAMS 3.5: MODULAR DESIGN 3.6: INPUT DESIGN 3.7: OUTPUT DESIGN	
4.	CHAPTER 4: SYSTEM REQUIREMENTS 4.1: INTRODUCTION	

	4.2: SOFTWARE REQUIREMENTS SPECIFICATION 4.3: HARDWARE REQUIREMENTS SPECIFICATION 4.4: TOOLS, PLATFORMS	
5.	CHAPTER 5: PROJECT IMPLEMENTATION 5.1: INTRODUCTION 5.2: CODING 5.3: DEBUGGING 5.4: UNIT TESTING	
6.	CHAPTER 6: SYSTEM TESTING 6.1: INTRODUCTION 6.2: INTEGRATION TESTING 6.3: SYSTEM TESTING 6.4: INTEGRATION TESTING	
7.	CHAPTER 7: SYSTEM MAINTENANCE 7.1: INTRODUCTION 7.2: MAINTENANCE	
8.	CHAPTER 8: SYSTEM SECURITY MEASURES 8.1: INTRODUCTION 8.1: OPERATING LEVEL SECURITY 8.2: DATABASE LEVEL SECURITY	

	8.3: SYSTEM LEVEL SECURITY	
9.	CHAPTER 9: SYSTEM PLANNING AND SCHEDULING 9.1: INTRODUCTION 9.2: PLANNING A SOFTWARE PROJECT 9.3: GANTT CHART 9.4: PERT CHART	
10.	CHAPTER 10: SYSTEM COST ESTIMATION 10.1: INTRODUCTION 10.2: LOC BASED ESTIMATION 10.3: FUNCTION POINT BASED ESTIMATION	
11.	CHAPTER 11: FUTURE ENHANCEMENT 11.1: INTRODUCTION 11.2: MERITS OF THE SYSTEM 11.3: LIMITATIONS OF THE SYSTEM 11.4: FUTURE ENHANCEMENT OF THE SYSTEM	
12.	CHAPTER 12: CONCLUSION	
13.	CHAPTER 13: BIBLIOGRAPHY	
14.	CHAPTER 14: APPENDIX	

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION ABOUT THE PROJECT

PropEase is a full-stack real estate platform developed using the MERN stack that allows users to list, search, rent, and buy properties with ease. The platform is designed to bridge the gap between property owners and seekers by offering a simple, intuitive, and interactive experience for all users.

The **user module**, built with **React.js** and styled using **Tailwind CSS**, enables users to explore property listings based on various filters such as location, price, and amenities. It supports a fully responsive layout, ensuring accessibility from both desktop and mobile devices. Users can also save listings for future reference and engage in real-time conversations with property owners through the built-in chat system powered by **Socket.io**.

The **owner module** allows landlords and property agents to register, list properties, upload images via **Cloudinary**, and manage their property details with complete control. The listing process is optimized for ease-of-use, encouraging more property owners to make their spaces available to potential buyers or tenants.

PropEase features secure authentication using **JWT (JSON Web Token)**, and backend operations are handled using **Node.js** and **Express.js**, while **MongoDB**, managed with **Prisma ORM**, serves as the database. The platform emphasizes user privacy, secure transactions, and performance-driven functionality, making it a modern solution for digital real estate interactions.

1.2 PROBLEM STATEMENT

In today's fast-paced digital world, the real estate industry still faces several challenges in connecting property owners with potential buyers and tenants through a unified online experience. Many existing platforms are either too complex for regular users or lack essential features such as real-time communication, secure authentication, and intuitive property discovery.

Users often struggle to find reliable platforms that offer up-to-date listings, filtering options based on specific needs (like location, budget, or amenities), and a seamless interface for direct interaction with property owners. At the same time, property owners and agents lack a user-friendly system where they can list, manage, and showcase properties effectively.

Moreover, many platforms do not offer personalized features like saved listings, chat-based inquiries, or location-aware browsing, limiting the decision-making capability of both

renters and buyers. There is also an increasing demand for scalable and responsive platforms that maintain performance across devices without sacrificing usability or speed.

There is a clear need for a dedicated, easy-to-use real estate platform that allows users to explore, rent, and buy properties, while also giving property owners full control over listing and engagement—without unnecessary complexity or third-party involvement.

1.3 SCOPE AND RELEVANCE OF THE PROJECT

The scope of the **PropEase** project is to develop a comprehensive, user-centric real estate platform that simplifies the processes of listing, renting, and buying properties. The system is designed to offer a seamless experience for both property seekers and property owners through modern UI/UX practices and scalable architecture. Users can browse properties using advanced filters, communicate in real-time with owners, and save listings for future consideration—all from an intuitive, responsive interface.

PropEase empowers property owners with an efficient listing system where they can upload property details and images, edit listings, and manage interactions with potential tenants or buyers without any third-party interference. The platform focuses on eliminating complexities found in traditional real estate websites by offering a clean, minimal, and mobile-friendly experience.

Developed using the MERN stack, PropEase utilizes **React.js** for the frontend, **Node.js** and **Express.js** for backend operations, and **MongoDB** with **Prisma ORM** for efficient database management. Features like **JWT-based authentication**, **Cloudinary integration for media management**, and **Socket.io for real-time chat** enhance usability and functionality, ensuring a smooth and secure user journey.

The relevance of PropEase lies in its ability to bring transparency and accessibility to real estate transactions. It bridges the communication gap between owners and seekers, enabling meaningful interactions without middlemen. By offering a digital-first, user-friendly solution, PropEase modernizes the property rental and sale process, catering especially to individuals who prefer convenience, control, and real-time engagement in property dealings.

1.4 OBJECTIVES

1. To develop a fully functional real estate web application that allows users to explore, rent, and buy properties through an intuitive and seamless interface.
2. To enable property owners to register, create and manage property listings, including uploading images, editing details, and responding to potential buyers or tenants.

3. To implement secure user authentication using JSON Web Tokens (JWT) to ensure privacy and protect user data across all interactions.
4. To provide advanced search and filtering capabilities that help users find suitable properties based on location, budget, property type, and other preferences.
5. To integrate a real-time chat system that facilitates direct communication between property owners and interested users, enhancing interaction and decision-making.
6. To ensure a responsive and scalable application through the use of modern technologies like React.js, Node.js, Express.js, and MongoDB with Prisma ORM.

SYSTEM ANALYSIS

CHAPTER 2

SYSTEM ANALYSIS

2.1 INTRODUCTION

System analysis is a critical phase in the development of any software application, as it lays the groundwork for understanding user requirements and translating them into an efficient and practical solution. In this chapter, we conduct an in-depth analysis of the current challenges faced in the real estate domain, including the inefficiencies in traditional property discovery, communication barriers between users and property owners, and the lack of centralized, user-friendly digital platforms.

By identifying the shortcomings in the existing system, this analysis will highlight the need for a modern, scalable, and accessible solution that caters to both property seekers and owners. The proposed system, **PropEase**, will be introduced as an enhanced platform that addresses these gaps with improved functionality, responsiveness, and real-time interaction features.

Additionally, a feasibility study will be performed to assess the technical, operational, and economic viability of the proposed solution, ensuring that the project is both implementable and sustainable. This will guide the development toward a solution that not only meets user expectations but also aligns with real-world constraints.

This chapter will also outline the software engineering paradigm adopted for the project, ensuring that the development process is systematic, well-structured, and adheres to modern best practices. Overall, this system analysis will form the foundation upon which the design, implementation, and deployment of PropEase will be carried out effectively.

2.2 EXISTING SYSTEM

In the current real estate market, most online property platforms are either too generic, limited in functionality, or overly complex for everyday users. Many existing systems focus solely on displaying listings without offering interactive or personalized features such as real-time chat, saved searches, or intuitive filters. These platforms often lack proper tools for direct communication between property seekers and owners, forcing users to rely on external channels to proceed with inquiries.

For property owners and agents, listing management is frequently tedious or restricted by third-party moderation and outdated interfaces. Uploading property details, managing listings, and updating content can become cumbersome, leading to delays in availability and reduced visibility of properties. Additionally, these platforms often charge fees or commissions, reducing transparency and trust in the process.

Moreover, existing platforms rarely provide location-aware features, dynamic filtering, or scalable backend support for growing data sets. The user experience is often hindered by outdated design, inconsistent data presentation, or a lack of responsive design for mobile devices.

Another significant limitation is the absence of real-time engagement tools. Without built-in messaging systems, users and property owners cannot engage in immediate dialogue, which is crucial in making informed rental or purchase decisions. There is a clear need for a modern, secure, and user-friendly platform that addresses these limitations allowing users to efficiently search, rent, and buy properties while empowering owners with control and flexibility in managing their listings.

2.2.1 Limitations of the Existing System

1. **Lack of Direct Communication:** Most existing real estate platforms do not support real-time messaging between property seekers and owners, forcing users to rely on external methods such as calls or emails, which delays responses and reduces engagement.
2. **Limited Property Management for Owners:** Property owners often have restricted control over editing, updating, or deleting listings. The absence of a user-friendly dashboard makes property management difficult and time-consuming.
3. **Poor Filtering and Search Functionality:** Many platforms provide only basic filtering options, which makes it difficult for users to find properties that match specific criteria like budget, location, property type, or amenities.
4. **No Save or Bookmark Feature:** Users are unable to save listings for future reference, requiring them to repeatedly search for properties they previously viewed—wasting time and reducing convenience.
5. **Outdated or Non-Responsive Design:** A significant number of platforms are not mobile-optimized or built with responsive design in mind, making it hard for users to navigate the interface on different devices.
6. **Lack of Geolocation Integration:** Users cannot view property locations interactively on a map or based on current geographic proximity, making it harder to evaluate locality and accessibility.
7. **No Personalization or User Dashboard:** Users often don't have personalized dashboards that track saved properties, interactions, or preferences—limiting the overall experience and functionality.

2.3 PROPOSED SYSTEM

The proposed **PropEase** system is a modern, scalable, and user-friendly real estate platform built using the **MERN stack**, designed to provide a seamless experience for both property

seekers and owners. It enables users to explore, list, rent, and buy properties through an intuitive interface that supports responsive interaction across web devices.

The **user module** is developed using **React.js** and styled with **Tailwind CSS**, offering a fast, interactive, and mobile-friendly experience. Users can browse listings using detailed search filters, view images, check location details, save properties for later, and communicate directly with property owners using the built-in real-time **chat system powered by Socket.io**.

The **owner module** is tailored for landlords and agents, allowing them to register, list properties with image uploads (handled by **Cloudinary**), edit or remove listings, and manage inquiries efficiently through a personal dashboard. This empowers property owners with complete control over their property listings and interactions, without relying on third-party agents. Backend operations are managed using **Node.js** and **Express.js**, while **MongoDB**, accessed via **Prisma ORM**, serves as the core database system. The platform uses **JWT-based authentication** to ensure secure user access and protect sensitive data throughout the system.

PropEase prioritizes ease of use and convenience through features such as advanced filtering, saved listings, and location-based property viewing. By eliminating unnecessary complexities, the platform focuses on providing a smooth and straightforward property search and management experience. By integrating powerful frontend technologies with a secure and scalable backend, PropEase delivers a holistic solution for real estate interactions—bridging the gap between property owners and seekers through real-time communication, personalized dashboards, and efficient listing management. The system stands out as a future-ready digital property hub tailored for simplicity, functionality, and engagement.

2.3.1 Advantages of the Proposed System

1. **User-Friendly Interface:** PropEase offers a clean, responsive, and intuitive user interface developed using React.js and Tailwind CSS, ensuring smooth navigation and ease of use across all devices.
2. **Real-Time Communication:** The built-in chat system allows users and property owners to communicate instantly, reducing delays and enhancing decision-making in the rental or buying process.
3. **No Third-Party Dependency:** Property owners can manage their listings independently adding, editing, or removing properties without relying on agents or middlemen.
4. **Secure Authentication:** JWT-based authentication ensures secure login and protects user data, providing a safe and trustworthy environment for all platform users.
5. **Advanced Property Search:** Users can apply filters such as location, budget, property type, and amenities to easily discover properties that match their exact requirements.
6. **Saved Listings:** Users can bookmark or save property listings for later reference, making it easier to track and compare options without repeated searches.

7. **Geolocation Support:** PropEase enables users to view property locations using latitude and longitude coordinates, making it easier to evaluate locality and neighborhood accessibility.
8. **Scalable Architecture:** Powered by the MERN stack with Prisma ORM, the system is designed for scalability and can handle a growing number of users and listings efficiently.

2.4 FEASIBILITY STUDY

A feasibility study is conducted to assess the viability of the *Rental Platform System*, ensuring that it can be successfully developed, deployed, and maintained within technical, operational, and economic constraints. This assessment aims to determine whether the platform can efficiently serve landlords and tenants while maintaining a seamless user experience, ensuring scalability, and staying within budgetary limits.

The feasibility study is divided into three core dimensions: technical feasibility, operational feasibility, and economic feasibility. These factors are critical in ensuring that the rental platform can meet its goals of connecting landlords and tenants for both renting and buying properties, while supporting a smooth, efficient, and cost-effective service.

2.4.1 TECHNICAL FEASIBILITY

The proposed *Rental Platform System* leverages modern technologies to provide a seamless and efficient service for connecting landlords and tenants for both renting and buying properties. The platform is designed to ensure a smooth user experience, scalability, and high performance, while maintaining simplicity and cost-effectiveness through its full-stack architecture.

The **user-facing module** is developed using **ReactJS**, which offers a responsive, high-performance web interface. Users can browse property listings, apply filters (such as location, price, and property type), and view detailed information about each property through an intuitive interface. The platform will allow users to create accounts, save their favorite properties, and directly communicate with landlords or agents through an integrated messaging feature.

The **admin module** is built using **HTML, CSS, and JavaScript**, providing a responsive and user-friendly web-based interface. Administrators have the ability to manage user profiles, approve or reject property listings, and ensure that all content meets platform guidelines. Admins can also manage the platform's content, including handling user queries and complaints.

The **backend** is powered by **Node.js** with the **Express.js** framework. This combination allows for an efficient and scalable RESTful API that handles client requests, manages data flow, and performs operations like user authentication, listing management, and messaging services. The backend communicates with **MongoDB**, a NoSQL database that is both flexible and scalable, allowing for the storage of user data, property listings, and

interactions between tenants and landlords. MongoDB's flexible schema is ideal for handling diverse data types, and its scalability ensures the platform can grow as the number of users and listings increases.

Security is a core aspect of the system. The platform employs **JWT (JSON Web Tokens)** for secure user authentication and authorization. This ensures that user data is protected and that only authorized users can access specific platform features. Passwords and sensitive data will be encrypted using industry-standard encryption techniques to ensure user privacy and security.

As a **full-stack** application, the platform integrates both the frontend and backend to provide a unified and efficient system. The backend manages all data processing, user management, and interactions, while the frontend focuses on delivering a seamless and responsive user experience. The application is hosted on a traditional web server with a focus on optimizing performance and ensuring uptime, without relying on cloud-based infrastructure.

The system is designed to be **scalable**, supporting future growth as the user base expands. While cloud services are not currently being used, the platform can still scale effectively through increased server resources and optimizing the application's performance.

Future enhancements might include adding features such as **real-time chat** for landlord-tenant communication and advanced **search filters** for property discovery, which can be added easily within the existing framework. The platform is also flexible enough to

2.4.2 OPERATIONAL FEASIBILITY

The successful implementation of the *Rental Platform System* depends on its ease of use, scalability, and low maintenance requirements. The platform is designed to offer a smooth, reliable experience for both users (landlords and tenants) and administrators, ensuring that the platform can be operated with minimal effort while delivering a seamless experience.

The **user interface**, developed using **ReactJS**, is highly intuitive and optimized for various devices. Users can easily browse property listings, apply filters, save favorite properties, and contact landlords—all without the need for specialized training. The responsive, mobile-friendly design ensures that the platform works consistently across different screen sizes and devices, making it accessible to a wide audience.

Landlords can **easily register**, list properties, manage their listings, and communicate with potential tenants through a simple web interface. They can also update property details and images, ensuring that their listings remain current. The platform simplifies the property management process for landlords, helping them reach a larger pool of potential tenants and buyers.

The **admin dashboard**, built with **HTML, CSS, and JavaScript**, provides a straightforward yet powerful interface for content moderation, user management, and

platform oversight. Administrators can approve or reject property listings, manage user profiles, and monitor interactions on the platform to ensure the quality of service. The admin tools are designed to minimize complexity, enabling administrators to effectively oversee the platform without requiring extensive training or technical expertise.

The platform is architected for **scalability**, allowing for the seamless addition of new features, user accounts, and property listings as the platform grows. The backend, powered by **Node.js** and **Express.js**, is capable of handling concurrent user interactions and managing large datasets efficiently. **MongoDB** serves as the database, providing a flexible and scalable solution to store user data, property details, and communication history.

Maintenance is minimal due to the modular structure of the platform. Regular updates, including bug fixes, feature improvements, and security patches, can be deployed easily. The system's design allows for straightforward updates and maintenance, with minimal downtime or disruption to users. Since the platform does not rely on cloud-based infrastructure, it can still be efficiently maintained using traditional hosting solutions, optimizing cost-efficiency.

Administrative tasks are simplified through **automated moderation tools** that help detect inappropriate content or user behavior, reducing manual intervention. Additionally, the platform's architecture ensures **high performance** and **reliability**, providing a consistent user experience even as the platform scales to accommodate more listings and users.

2.4.3 ECONOMIC FEASIBILITY

The economic viability of the Rental Platform System is crucial for its long-term success, scalability, and wide adoption. The platform is designed to be cost-effective, leveraging open-source technologies, affordable hosting solutions, and a streamlined system architecture to deliver a high-quality user experience while keeping operational costs minimal.

The initial investments primarily cover the setup of servers, databases, and deployment infrastructure. The platform uses traditional web hosting services, ensuring that the cost structure remains manageable without the need for expensive cloud-based services. These hosting solutions offer pay-as-you-go pricing models, which align costs with usage, ensuring that operational expenses remain proportional to the platform's growth.

The use of **Node.js**, **Express.js**, and **MongoDB**—all open-source technologies—greatly reduces software licensing costs. The platform's backend architecture, coupled with **ReactJS** for the frontend, is designed to be lightweight, efficient, and maintainable, optimizing both development and ongoing operational expenses. The use of **ReactJS** ensures that the platform is highly modular and reusable, allowing for easier and more cost-effective updates and feature additions.

In terms of operational overhead, the platform minimizes recurring costs through efficient backend development and automation tools. The user and admin management processes are automated, and content moderation can be handled by lightweight tools and scripts, reducing the need for manual intervention. Additionally, the platform's architecture is designed to optimize server usage, reducing the overall cost of maintenance while ensuring high availability and performance.

Revenue generation for the Rental Platform System can come from several monetization strategies, including:

- **Service fees:** Charging landlords a small fee for listing properties, which can be a flat rate or a percentage of the final transaction (rental or sale).
- **Premium listings:** Offering landlords the option to pay for enhanced visibility of their property listings, increasing their chances of attracting tenants or buyers.
- **Subscription models:** Offering users a subscription to access additional features such as saving multiple property searches, advanced filtering options, or receiving early notifications for new listings.
- **Advertising:** Integrating third-party ads on the platform, which can be particularly beneficial for generating revenue from free-tier users.
- **Partnerships with real estate agents and agencies:** Allowing agents to list properties in bulk for a subscription or pay-per-listing fee.

These revenue streams ensure the platform's long-term sustainability and profitability, allowing it to reinvest into development, marketing, and user acquisition. Furthermore, by offering free access to users, the platform can maintain a large user base, while generating income through the monetization strategies targeted at landlords and agents.

The platform's **scalable architecture** enables it to launch with a modest user base and expand as needed. Its modular design supports future feature additions, such as advanced property search, enhanced user profiles, and third-party integrations, without requiring significant changes to the infrastructure. This scalability ensures that the platform can grow gradually, spreading out operational costs and maximizing profitability as demand increases.

In conclusion, the Rental Platform System presents a cost-effective solution for landlords and tenants, with a strong potential for long-term growth and profitability. Its use of open-source technologies and affordable hosting ensures that both development and operational costs remain low. The diverse revenue models provide opportunities for monetization, while the platform's scalability and modularity make it economically viable in both the short and long term. By providing a free and accessible service to users while generating revenue from landlords and real estate agents, the platform offers a competitive and sustainable business model in the growing real estate market.

2.5 SOFTWARE ENGINEERING PARADIGM APPLIED

The software process refers to the method by which software is produced, incorporating principles that ensure the development of reliable, efficient, and economically feasible software. For the development of the Rental Platform System, the chosen software engineering paradigm is the **Agile Methodology**, which is iterative, flexible, and allows for continuous improvement and adaptation to changes in project requirements.

The **Agile Model** emphasizes collaboration, flexibility, and customer feedback, which aligns well with the needs of a rental platform system where user requirements might evolve over time. Agile's iterative approach helps in developing small increments of the system, ensuring that the platform can evolve with ongoing feedback from stakeholders, including landlords, tenants, and administrators.

For the Rental Platform System, the Agile methodology ensures that the development process remains adaptive and responsive to changes, while also allowing for constant testing and validation of features. This model supports the addition of new functionalities such as property filters, user messaging systems, and subscription features, with continuous testing and iteration to ensure high quality and customer satisfaction.

Below are the key phases of the **Agile Methodology** as applied to the Rental Platform System:

1. Requirement Analysis Phase

In this initial phase, the requirements for the *Rental Platform System* are gathered and documented. This includes:

- **Core Features:** The essential features such as user registration, property listings, search functionality, messaging system, and user management. These features must be clearly defined for both tenants and landlords.
- **Hardware Components:** Identifying necessary infrastructure components such as databases (MongoDB for flexible, document-based storage), web servers, and any external services for notifications or geolocation-based services.
- **Software Requirements:** Detailed specifications for the front-end and back-end systems. The front-end will be developed using ReactJS for a dynamic user interface, while the back-end will rely on Node.js to handle business logic and data manipulation. Additionally, MongoDB will be used to manage data such as user profiles, property listings, and user interactions.
- **Stakeholders:** Identifying key stakeholders including tenants, landlords, administrators, and developers, each of whom will interact with different aspects of the platform.

2. System Design Phase

Based on the gathered requirements, the system architecture is designed. This phase includes:

- **Hardware Design:** Selecting MongoDB as the primary database for its scalability and flexibility in handling diverse datasets, such as user profiles, property listings, and transaction records. Servers and hosting services are also planned to handle anticipated traffic, ensuring seamless user access.
- **Software Design:**
 - **Front-End Design:** The user interface for both tenants and landlords will be designed using ReactJS, ensuring responsive and dynamic interactions.
 - **Back-End Design:** The back-end system architecture will be based on Node.js, handling API requests, user management, and property listing functionalities.
 - **System Integration:** Integration between Node.js and MongoDB to handle data storage and retrieval, ensuring that operations like user authentication, property listing, and messaging can be processed smoothly. Special attention will be given to the design of RESTful APIs to facilitate communication between the front-end and back-end systems.

3. Implementation Phase

In this phase, the actual development of the *Rental Platform System* takes place. The implementation involves:

- **Programming and Coding:** The core functionalities for property management, search filters, messaging systems, user registration, and authentication will be developed using ReactJS for the front-end and Node.js for the back-end. MongoDB will be used to store user information, property data, and interactions.
- **Integration of Components:**
 - The front-end will interact with the back-end through APIs, exchanging user information, property details, and other real-time data.
 - The back-end will integrate with MongoDB for storing and retrieving property listings, user profiles, and messages.
- **Optimization for Performance:** During this phase, attention will be given to optimizing database queries, especially those related to property searches, user interactions, and messaging systems. Indexing frequently queried fields in MongoDB will be done to improve data retrieval speed.

4. Testing Phase

Once the system is developed, it undergoes rigorous testing to verify functionality, performance, and security. Testing includes:

- **Unit Testing:** Each module, including user authentication, property listing, and the messaging system, will be tested individually to ensure proper functionality. Tests will also include database interactions, especially CRUD operations with MongoDB.
- **System Testing:** Ensuring that all modules work together as expected, with a focus on the performance of the database during real-time property searches and user interactions. Load testing will be conducted to simulate high traffic conditions.
- **User Acceptance Testing (UAT):** Feedback from users (tenants, landlords, and administrators) will be collected to ensure the platform meets their needs and provides a smooth experience. Testing will also focus on the efficiency and reliability of database interactions with MongoDB.

5. Deployment Phase

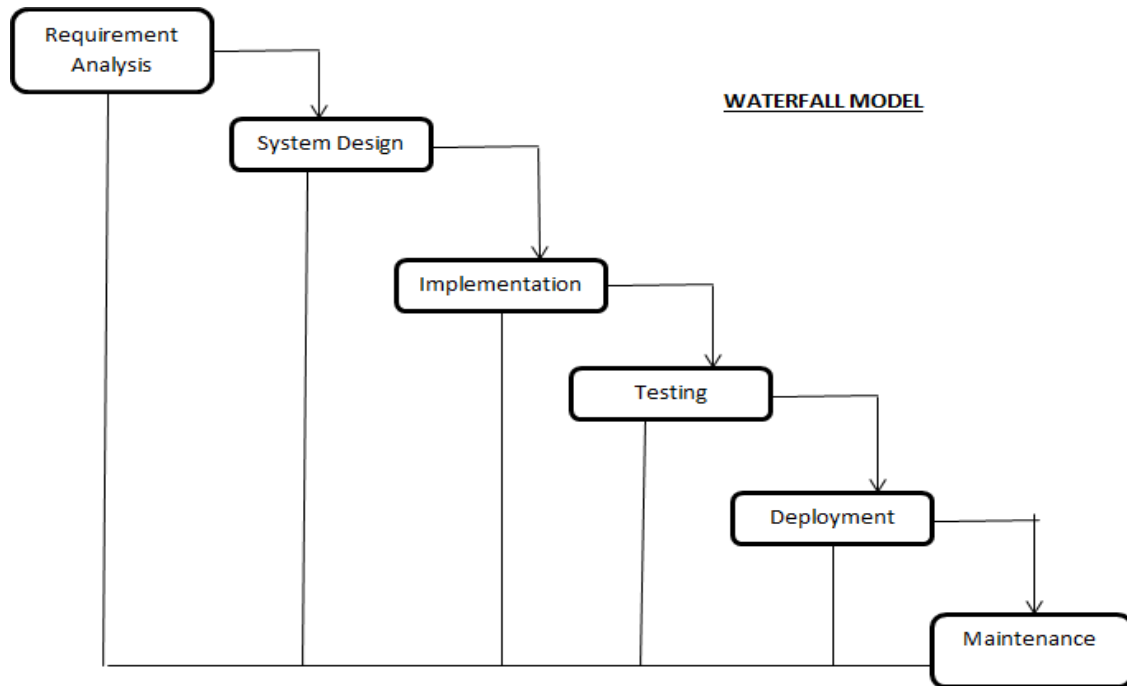
After successful testing, the system is deployed for real-world use. The deployment process includes:

- **Setting up Servers and Databases:** The necessary hosting infrastructure for both the front-end and back-end will be set up, along with MongoDB to handle real-time data processing and storage.
- **Launching the Application:** The web application will be deployed, ensuring seamless access for users and administrators. The system will connect to MongoDB for dynamic content management, user profiles, and property listings.
- **Administrator Training:** Administrators will be trained on managing content, handling user issues, and overseeing the platform's operations. They will also be trained on managing the MongoDB database, including query optimization and data backup procedures.

6. Maintenance Phase

Ongoing maintenance is required to ensure the *Rental Platform System* runs smoothly and continues to meet user demands. Maintenance activities include:

- **Regular Updates:** The platform will undergo updates to introduce new features, improve performance, and resolve any identified issues. Special focus will be placed on optimizing MongoDB queries, particularly as the database grows and more data is added.
- **Content Management and Data Handling:** Continuous management of property listings, user data, and other platform content. The MongoDB database will be optimized regularly to handle higher volumes of data as the platform scales.
- **System Upgrades:** As the platform evolves, significant updates to the database schema may be needed. This will require careful planning, testing, and deployment to avoid disruptions in service.



Advantages of the Waterfall Model

- 1 **Simple and Easy to Understand:** The linear approach ensures clarity in development stages.
- 2 **Clear Structure:** Each phase has defined deliverables, ensuring organized project execution.
- 3 **Documentation:** Comprehensive documentation ensures knowledge transfer and long-term maintainability.
- 4 **Effective for Stable Requirements:** Ideal for projects with well-defined objectives, such as air quality monitoring.
- 5 **Ease of Management:** The structured process allows efficient tracking of progress and resource allocation.
- 6 **Early Identification of Requirements:** Complete requirement gathering at the start minimizes risks of missing critical functionalities.

SYSTEM DESIGN

CHAPTER 3

SYSTEM DESIGN

3.1 INTRODUCTION

The system design phase of the *Rental Platform System*, a comprehensive web application for connecting tenants and landlords, is focused on transforming conceptual requirements into a well-structured, scalable, and user-friendly architecture. This phase ensures efficient data handling, reliable search functionalities, and an intuitive user experience for tenants, landlords, and administrators.

The *Rental Platform System* is designed to support core features such as property listings, advanced search filters, property details, user registration, messaging between tenants and landlords, and real-time updates. It ensures high performance, security, and scalability across both web and mobile platforms.

The architecture follows a modular and role-based design:

- **Tenant and Landlord Modules:** The user interface for tenants and landlords is built using **ReactJS**, providing a dynamic and responsive experience across web browsers. This ensures users can easily interact with the platform, browse properties, and manage their accounts.
- **Admin Interface:** The admin interface is also built using **ReactJS**, allowing administrators to manage user profiles, listings, messages, and general platform settings. The interface is designed for easy use and seamless navigation.
- **Back-End Logic:** The backend of the platform is powered by **Node.js** with **Express.js**, which handles API requests, user authentication, and the core business logic of property management, messaging, and search functionality.
- **Database:** **MongoDB** serves as the primary database, managing structured and unstructured data such as user profiles, property listings, transaction records, and user interactions. Its flexibility in handling diverse data types makes it ideal for scaling with the platform's growth.

To ensure secure access and content management, the platform employs **role-based access control (RBAC)**, assigning different permission levels to tenants, landlords, and administrators. User authentication is managed through **JWT (JSON Web Tokens)**, ensuring protected access to user-specific data and actions.

The system communicates between the frontend and backend through **RESTful APIs**, promoting modularity and simplifying future upgrades or third-party integrations. The

platform is designed to scale horizontally, allowing for the addition of microservices or cloud-based resources as the platform grows, ensuring the system can handle increased traffic and data.

Through careful planning of hardware resources, database schema design, software layers, and system interactions, the system design phase establishes the foundation for a reliable, secure, and user-centric rental platform. The *Rental Platform System* is engineered to provide a seamless experience for tenants and landlords while empowering administrators to efficiently manage platform operations.

3.2 DATABASE DESIGN

The database design of the *Rental Platform System* is integral to ensuring secure, efficient, and scalable data management across the platform. It is structured to handle critical information related to tenants, landlords, property listings, transaction history, and messaging, guaranteeing a smooth and reliable experience for all users, including tenants, landlords, and administrators.

Built using **MongoDB**, the database follows a NoSQL model, which allows for flexible data storage, quick retrieval, and scalability. Given the dynamic nature of real estate data (including multimedia content, property details, and tenant-landlord interactions), the NoSQL approach ensures that data is stored efficiently, and scalability is easily achievable as the platform grows.

The **Entity-Relationship (ER) model** is designed to represent the key entities and their relationships:

- **Users:** Tenants and landlords have profiles that include personal information, preferences, transaction history, and messages. Tenants can also have saved properties and favorites.
- **Properties:** Each property listing includes detailed information such as property type, location, price, size, amenities, photos, and availability. These listings can be linked to tenants and landlords for management.
- **Messages:** Tenants and landlords can interact via an internal messaging system. This data includes timestamps, message content, and associated property or user profiles.
- **Transactions:** Logs of rental agreements and potential future transactions are stored to track payments, agreements, and booking histories.
- **Admins:** Administrators manage platform functions, such as moderating listings, user accounts, and managing disputes or content issues. Admin privileges are stored separately to ensure data access control.

The database schema is designed to handle both structured data (user profiles, transactions) and unstructured data (property photos, messages). The flexible nature of **MongoDB** allows the platform to easily store varying information, from basic details to rich media like photos and videos, without requiring rigid table schemas.

The **ER model** integrates with a **data dictionary**, ensuring that every attribute in the database has a clear, consistent definition and is linked correctly to other entities. This approach facilitates accurate data storage, integrity, and retrieval, ensuring minimal redundancy and optimized performance.

3.2.1 Entity Relationship Model

Database designs also includes ER (Entity-Relationship model) diagrams. An ER diagram is a diagram that helps to design databases in an efficient way. Attributes in ER diagrams are usually modelled as an oval with the name of the attribute, linked to the entity or relationship that contains the attributes. Within the relational model, the final step can generally be broken down into two further steps that of determining what are the basic objects about which information, is being stored and then determining the relationships, or objects. This step is not necessary with an object database.

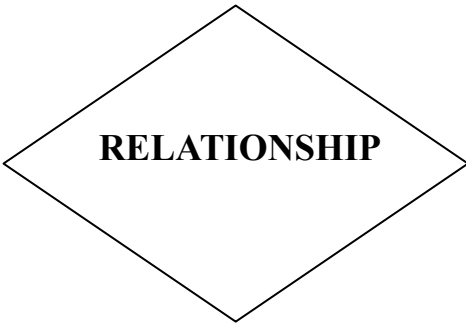
There are 5 main components of an ERD:

- Entities, which are represented by rectangles. An entity is an object or concept about which you want to store information. A weak entity is an entity that must defined by a foreign key relationship with another entity as it cannot be uniquely identified by its own attributes alone.



ENTITY

- Relationships, which are represented by a diamond shapes, show how to entities share information in the database. In some supervise other employee.



RELATIONSHIP

- Attributes, which are represented by ovals. A key attribute is the unique, distinguishing characteristic of the entity. For example, employee's social security number might be the employee's key attribute.



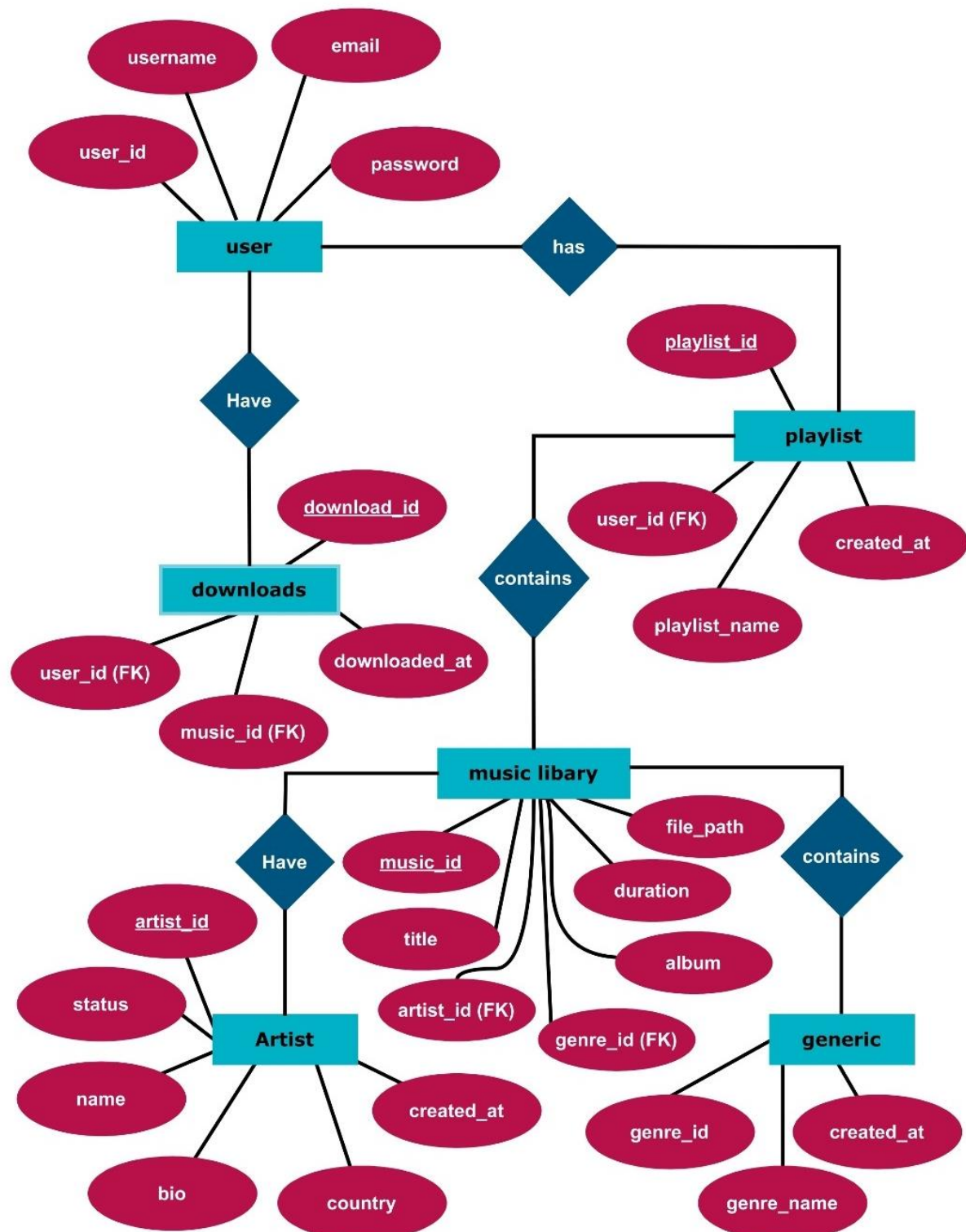
ATTRIBUTES

- Connecting lines, solid lines that connect attributes to show the relationships of entities in the diagram.
-

Steps involved in creating an ERD include:

1. Identifying and defining the entities.
2. Determining all interactions between the entities.
3. Analysing the nature of interactions/determining the cardinality of the relationship.
4. Creating the ERD.

E-R diagram shows the different entities in the system and the relationship among the different entities. Each entities have its own attributes associated with it; the relationship shows the association among the different entities.



3.2.2 Data Dictionary

1. user Table

Field	Type	Description
id	string (PK)	Unique identifier for the user
username	string	Username of the user
email	string	Email address
avatar	string	Profile picture URL
createdAt	Date	Account creation date

2. post Table

Field	Type	Description
id	string (PK)	Unique identifier for the post
userId	string	Foreign key to user (post creator)
title	string	Title of the property listing
price	int	Price of the property
images	string[]	Array of image URLs
address	string	Address of the property
city	string	City name
bedroom	int	Number of bedrooms
bathroom	int	Number of bathrooms
latitude	string	Latitude coordinate
longitude	string	Longitude coordinate
type	string	Rent or Sale
property	string	Property type (e.g., house, apartment)
createdAt	Date	Listing creation date

3. postDetail Table

Field	Type	Description
id	string (PK)	Unique ID for post detail record
postId	string	Foreign key to post table
desc	string	Description of the property
utilities	string	Utilities included
pet	string	Pet allowed (Yes/No)
income	string	Minimum required income
size	int	Property size (sq ft/m ²)
school	int	Distance to nearest school (in meters/km)

bus	int	Distance to nearest bus stop
restaurant	int	Distance to nearest restaurant

4. savedPost Table

Field	Type	Description
id	string (PK)	Unique ID for the saved post record
userId	string	Foreign key to user table
postId	string	Foreign key to post table
createdAt	Date	Date when post was saved

5. message Table

Field	Type	Description
id	string (PK)	Unique identifier for message
userId	string	Sender's user ID
chatId	string	Identifier for the chat
text	string	Message content
createdAt	Date	Time message was sent

3.3 DATAFLOW DIAGRAM

A Data Flow Diagram (DFD) provides a visual representation of how data flows through the Rental Platform System, capturing the interaction between users, posts, services, and the backend. It illustrates the journey of data from when a user registers or posts a property to when a potential tenant or buyer interacts with the listing. This ensures a clear understanding of the system's logical flow, helping identify the role of each component.

The DFD for the Rental Platform System demonstrates the end-to-end data journey — from user registration, property listing creation, and property browsing, to saved posts and messaging between users. It highlights the communication between the frontend interface, backend APIs, authentication, and database layers.

Key system components in the SpotifyLite DFD include:

Users (Landlords & Tenants):

- Landlords can register, post property listings for rent/sale, and manage their listings.

- Tenants/Buyers can browse available properties, view details, save listings, and contact landlords.

User Interface (React-based Web App):

- Provides users with interactive pages for login, registration, listing creation, property search, and detailed views.
- Accepts user input (form submissions, filters, messages) and displays personalized content and search results.

Authentication Module (JWT-based Auth):

- Validates user credentials during login and registration.
- Issues and manages JWT tokens to ensure secure access to user-restricted features like post creation and chat.

Database (MongoDB):

- Stores data such as user profiles, property listings, detailed post attributes, saved posts, and chat messages.
- Handles relationships between collections like user, post, postDetail, savedPost, and message.

Content Management & Post Handling:

- Allows users to create posts with media (images), location data (latitude/longitude), and property specifications.
- Stores metadata such as type (rent/sale), amenities, and nearby places.

Saved Post & Favorite Logic:

- Enables tenants to save interesting properties to view later.
- Ensures proper relational mapping between users and their saved listings.

Messaging System:

- Lets users communicate regarding a listing.
- Stores chat messages associated with post IDs and users for better conversation tracking.

Admin Panel (Future Scope):

- An admin dashboard (optional future enhancement) could be used for content moderation, user management, and platform analytics.

Five rules for constructing a Data Flow Diagram

- ❖ Arrows should not cross each other.
- ❖ Squares, circles and files must bear names.

- ❖ Decomposed data flow squares and circles can have same names.
- ❖ Choose meaningful names for data flow.
- ❖ Draw all data flows around the outside of the diagram.

DFD Symbols

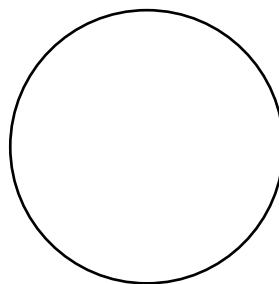
- ❖ A Rectangle defines the source or destination of system data.



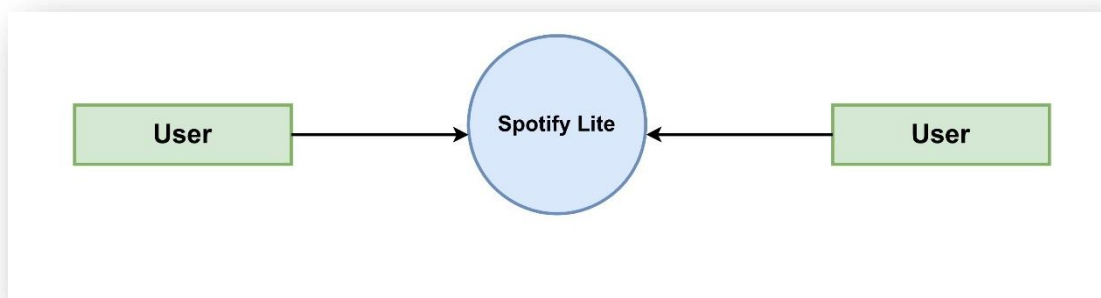
- ❖ An Arrow identifies flow of data in motion. It is a pipeline through which information flows.



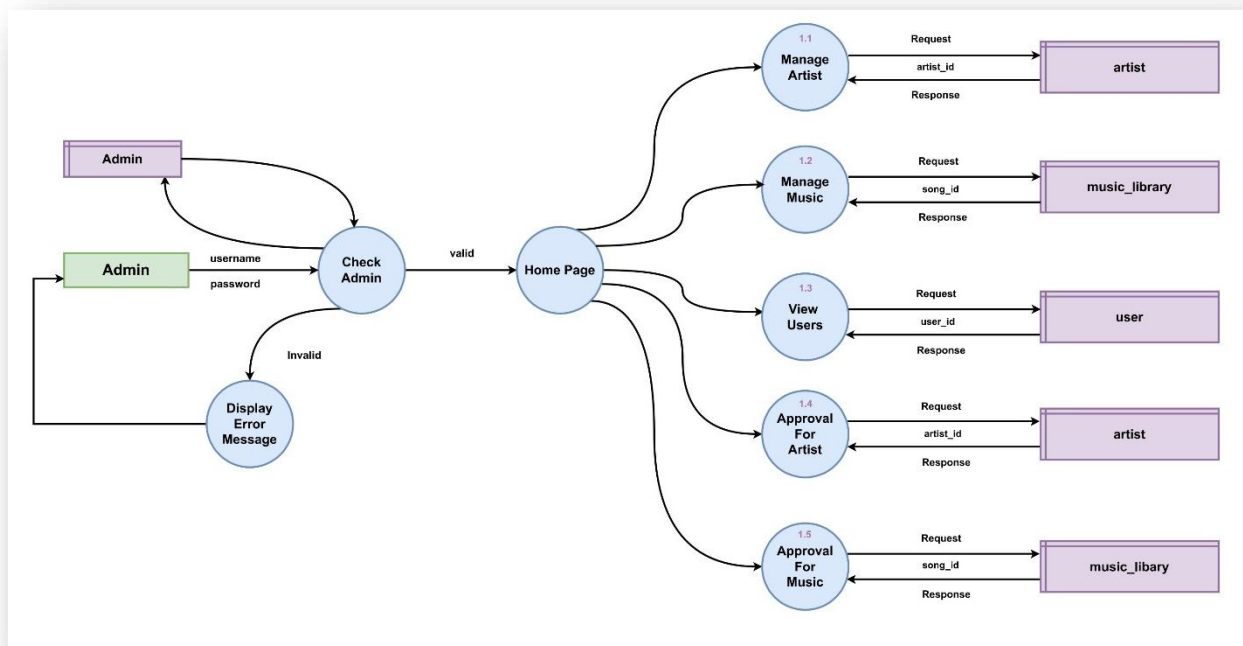
- ❖ A Circle or bubble represents a process that transforms incoming data flow into outgoing data flows.



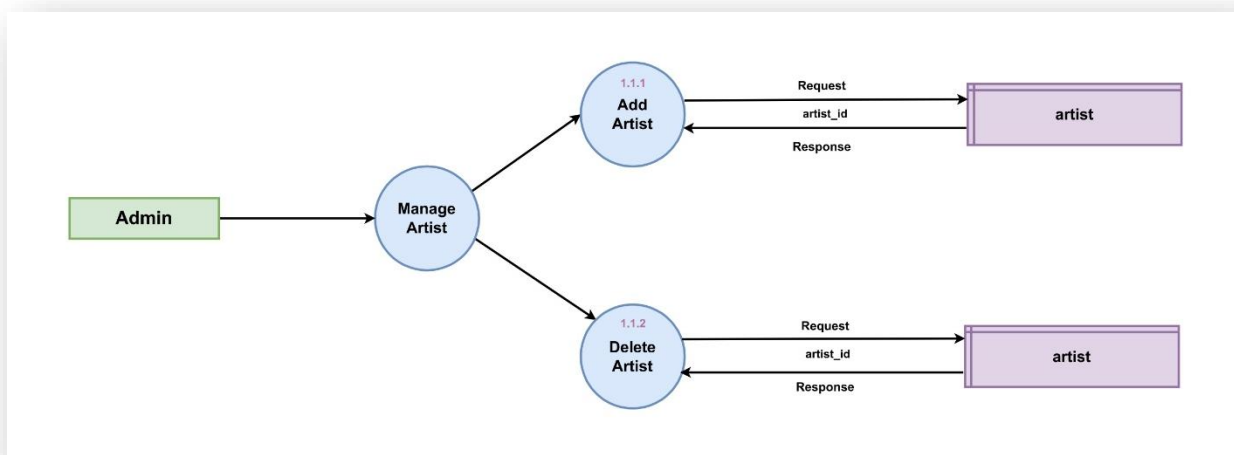
Level 0



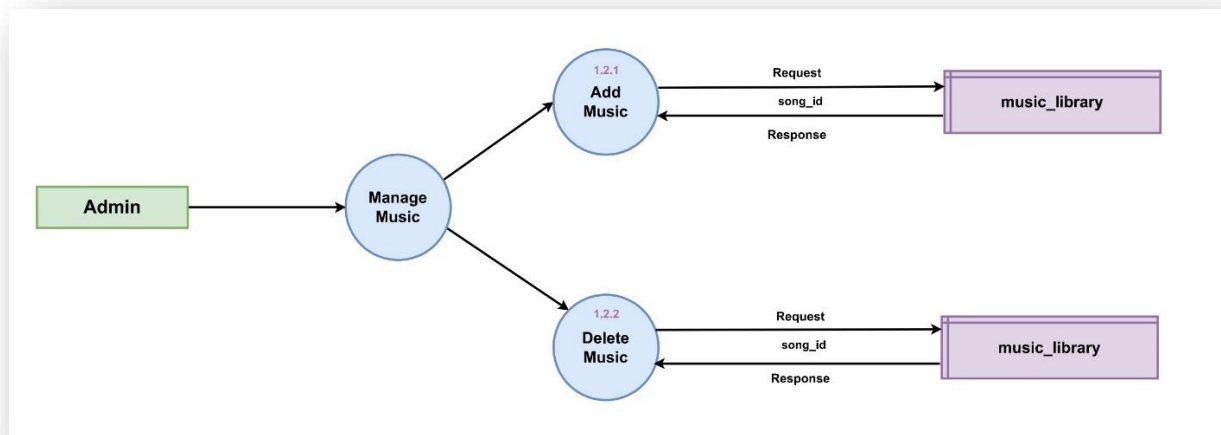
Admin Level 1



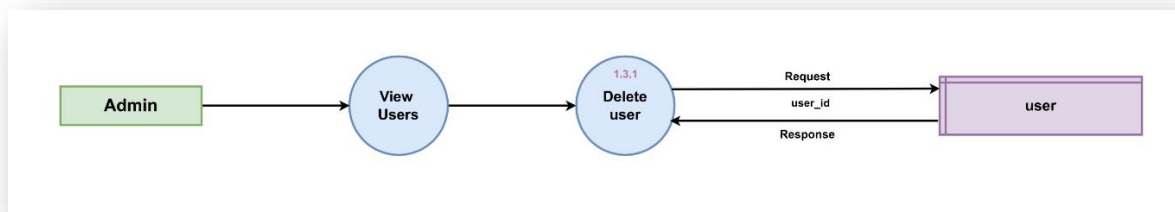
Admin Level 1.1



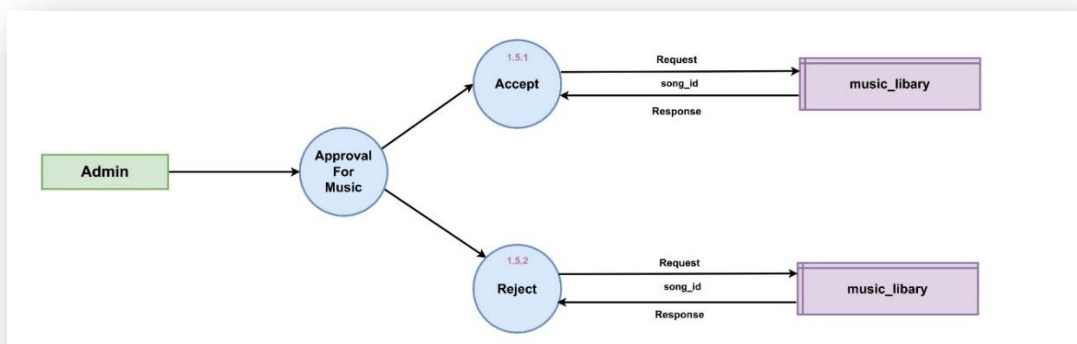
Admin Level 1.2



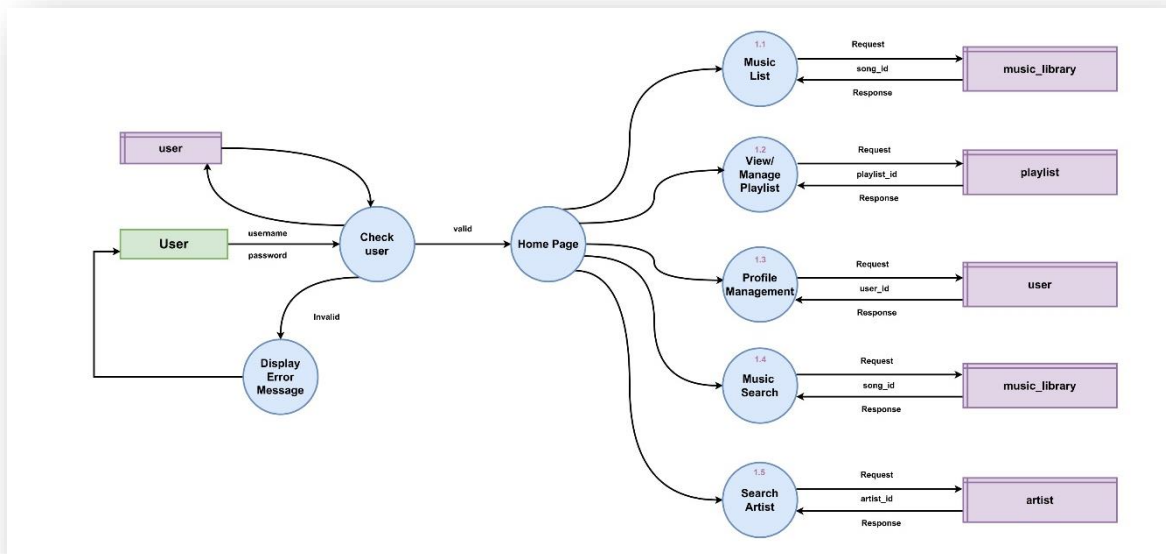
Admin Level 1.3



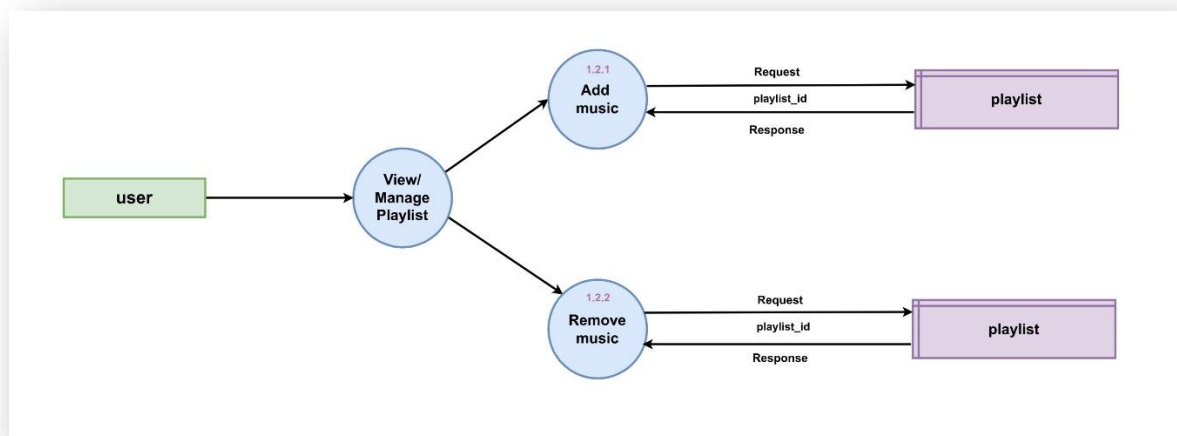
Admin Level 1.5



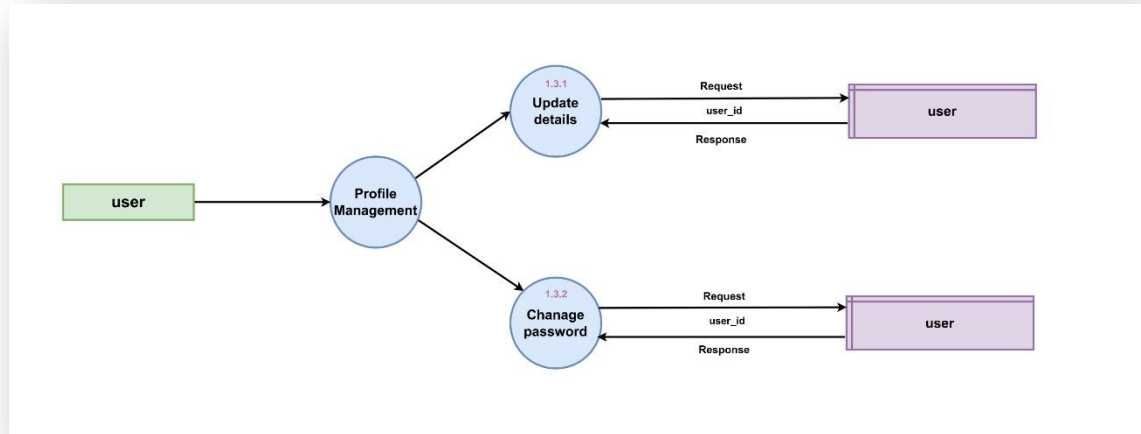
User Level 1



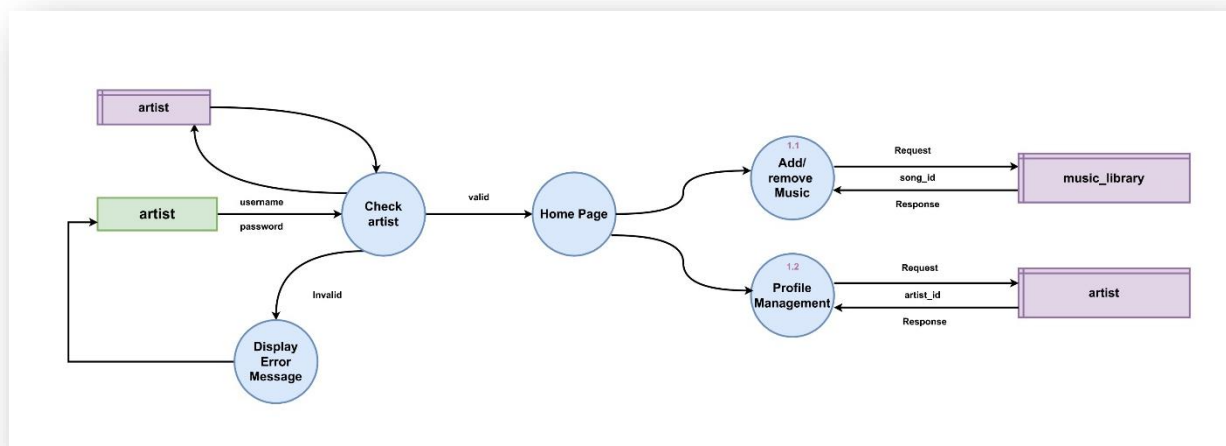
User Level 1.2



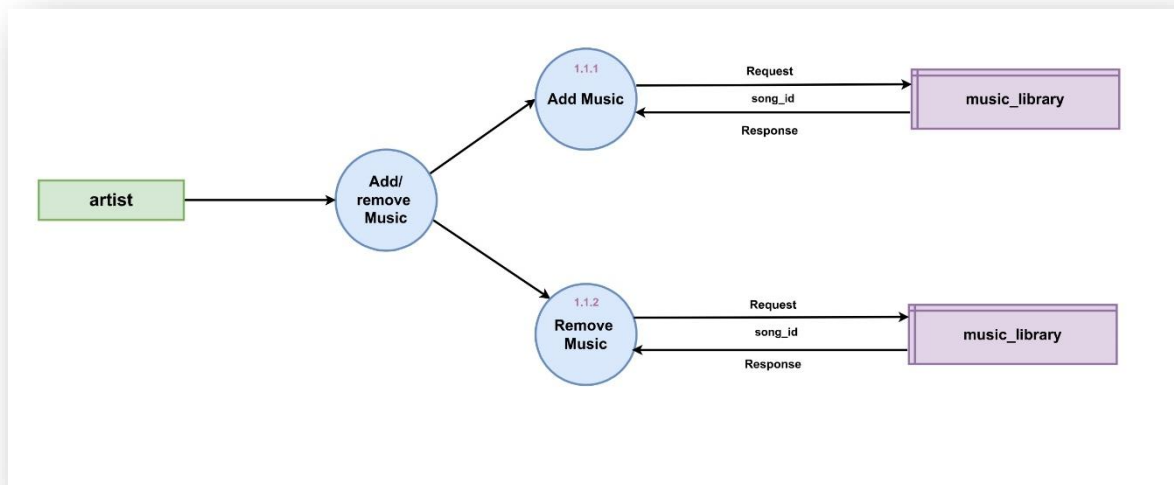
User Level 1.3



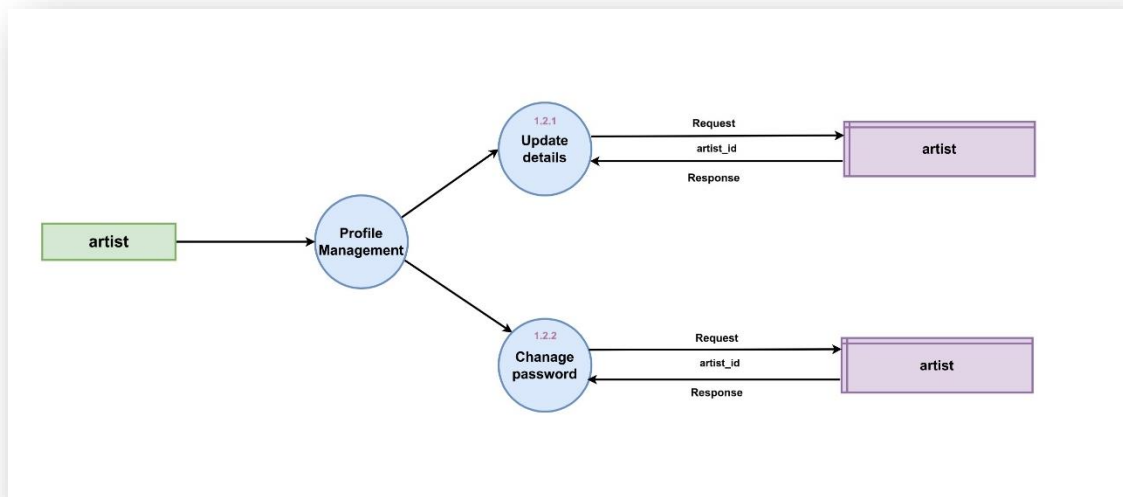
Artist Level 1



Artist Level 1.1



Artist Level 1.2



3.4 OBJECT ORIENTED DESIGN – UML DIAGRAMS

The UML (Unified Modeling Language) diagrams for the Rental Platform System provide a structured visualization of the application's architecture and behavior. These diagrams illustrate the interaction between users (landlords and tenants/buyers), components, and backend services, ensuring a clear roadmap for development and future scalability.

The structural aspect is represented through class diagrams, showcasing core entities such as User, Post, PostDetail, SavedPost, Message, and Chat. Relationships like association and composition define how users create property posts, add detailed property data, and engage in messaging. Each class includes attributes and key methods relevant to platform functionality, such as user authentication, property post creation, and real-time communication.

The behavioral aspect of the UML design focuses on the dynamic interactions between system components. The use case diagram identifies primary actors—Landlords, Tenants/Buyers—and their interactions including registration, login, post creation, property search, saving listings, and sending messages.

The sequence diagrams model key flows such as logging in, creating a property post, and initiating a chat between a tenant and a landlord. These diagrams describe how front-end inputs trigger backend processes and result in responses like displaying listings or sending messages via WebSocket.

Activity diagrams visualize essential workflows like user registration, property searching, saving posts, and chatting. State diagrams reflect status changes in system entities, such as a post moving from “draft” to “published,” or user sessions moving from “logged out” to “active.” Together, these UML diagrams provide a comprehensive and structured overview of the Rental Platform System's design. They help developers, testers, and stakeholders understand how various modules interact and evolve over time, supporting a modular, scalable, and user-friendly system.

3.4.1 Use Case Diagram

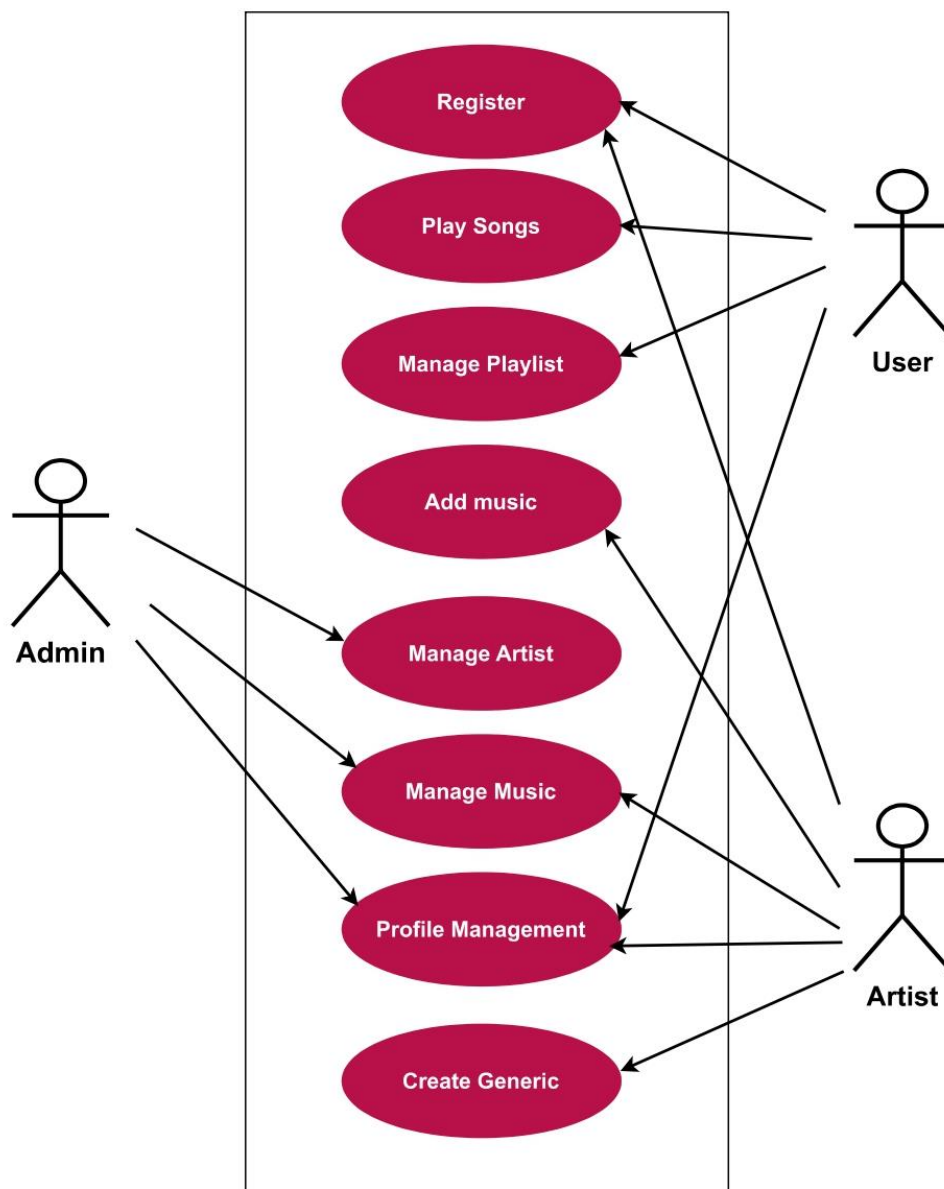
The Rental Platform System is designed to support three primary types of users: **Landlords**, **Tenants/Buyers**, and **Admins**. Each type of user interacts with the platform differently based on their roles, ensuring a seamless experience in searching, posting, and managing properties.

For **Tenants/Buyers**, the main interactions involve **searching for properties** based on filters such as location, budget, or property type. Users can **view property details**, **save posts** to revisit later, and **initiate chats** with landlords to inquire about the listings. To access these features, users must first **register** and **log in** to their accounts, after which they can manage their saved listings and chat conversations.

Landlords are the property owners or agents who can **create new property posts** by uploading relevant information and images. They can **edit or delete posts**, **manage their**

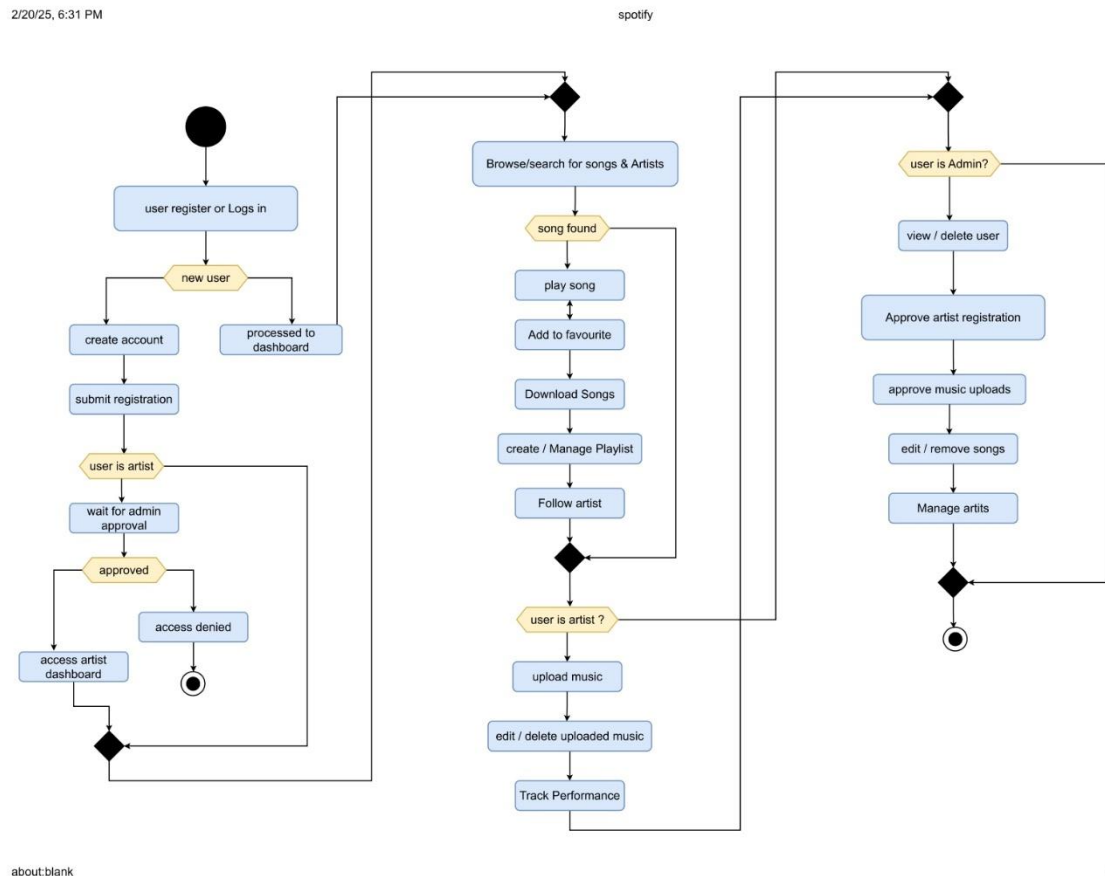
profile, and **chat with interested tenants/buyers** to provide more details or schedule visits. These users also need to **register** and **log in** before accessing posting features.

Admins are responsible for **managing the platform's integrity**. They have privileges to **review property posts, approve or reject them** to ensure quality and compliance. Admins can also **manage user accounts**, monitor chats for policy violations, and **oversee platform performance** to maintain smooth operations and a secure environment for all users.



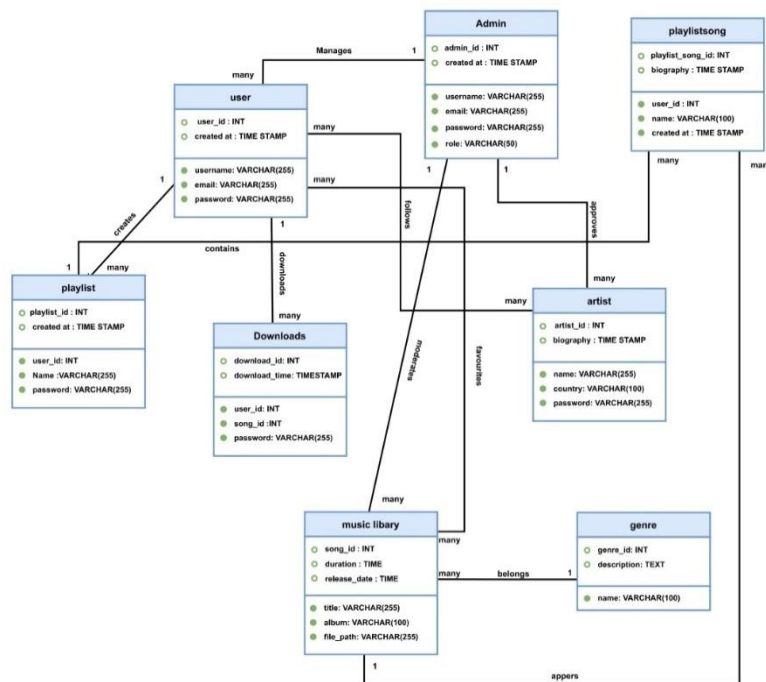
3.4.2 Activity Diagram

An activity diagram is a graphical representation of the flow of activities within a system, showcasing the sequence of tasks, decisions, and interactions among various components. It helps visualize the step-by-step execution of processes, making it easier to understand system behavior and workflows. By using elements like start and end nodes, actions, decision points, and parallel activities, the diagram effectively highlights dependencies, potential bottlenecks, and inefficiencies. This is particularly useful in process optimization, system modeling, and workflow analysis, ensuring a structured and efficient system design.



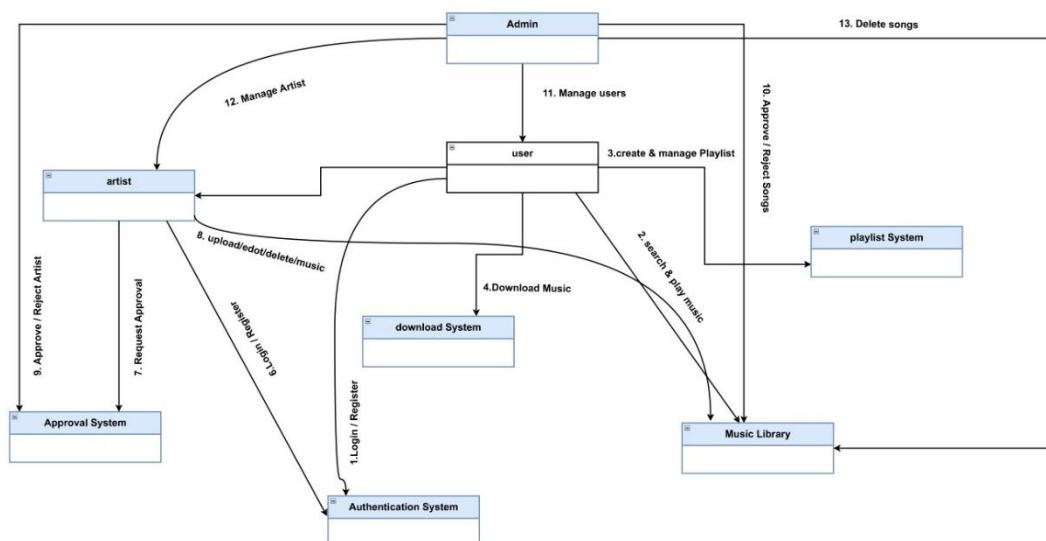
3.4.3 Class Diagram

A class diagram is a key component of object-oriented system design, visually representing the structure and relationships between classes, including their attributes, methods, and associations. It serves as a blueprint for software architecture, enabling developers to understand how different components interact within a system. By illustrating inheritance, composition, and dependencies, class diagrams enhance clarity, streamline development, and facilitate communication among team members. This structured representation helps in designing scalable, maintainable, and efficient software systems.



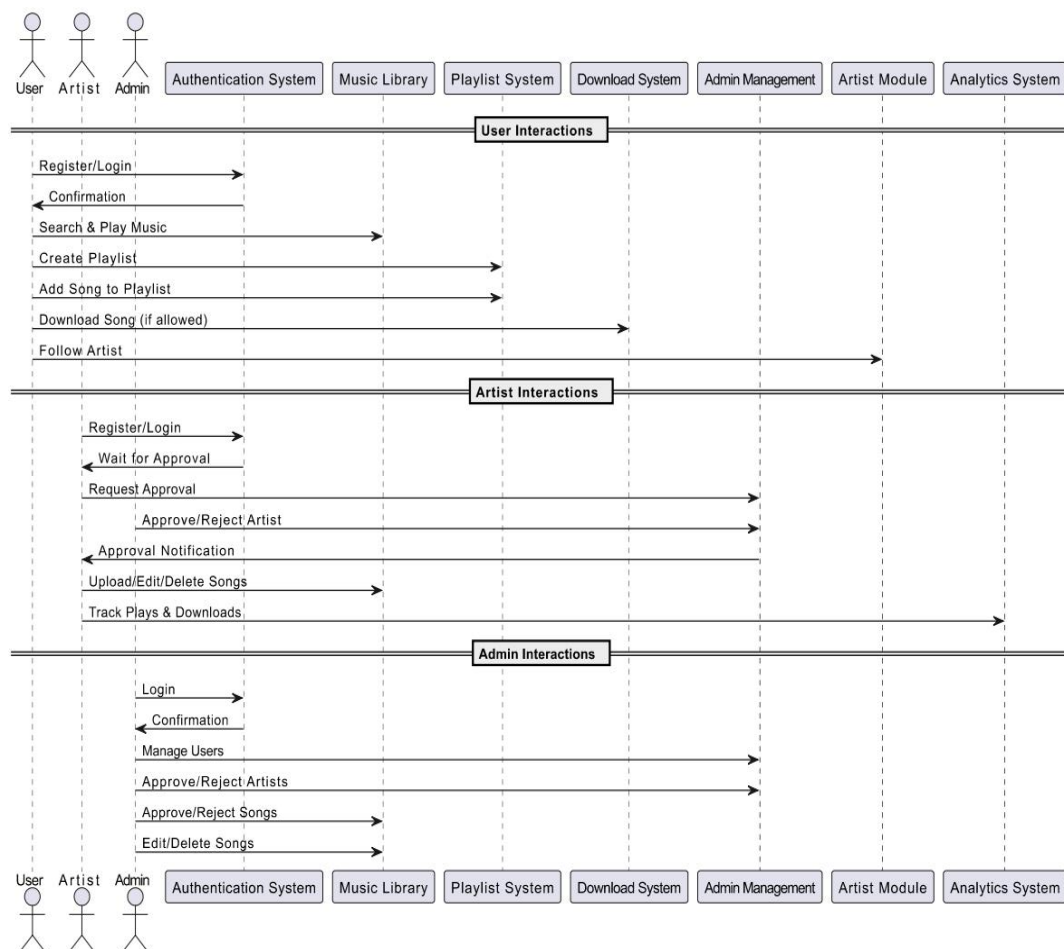
3.4.4 Communication Diagram

A Communication Diagram is a UML diagram that shows how objects in a system interact by exchanging messages to achieve a specific task. It highlights the relationships between objects and the flow of messages, with arrows indicating the sequence of interactions. Unlike sequence diagrams, which focus on the order of messages, communication diagrams emphasize object collaboration and their dependencies. Each object is represented as a rectangle, and the messages are numbered to show the sequence. These diagrams are helpful for understanding complex interactions and object relationships in a system, offering a high-level view of communication between components.



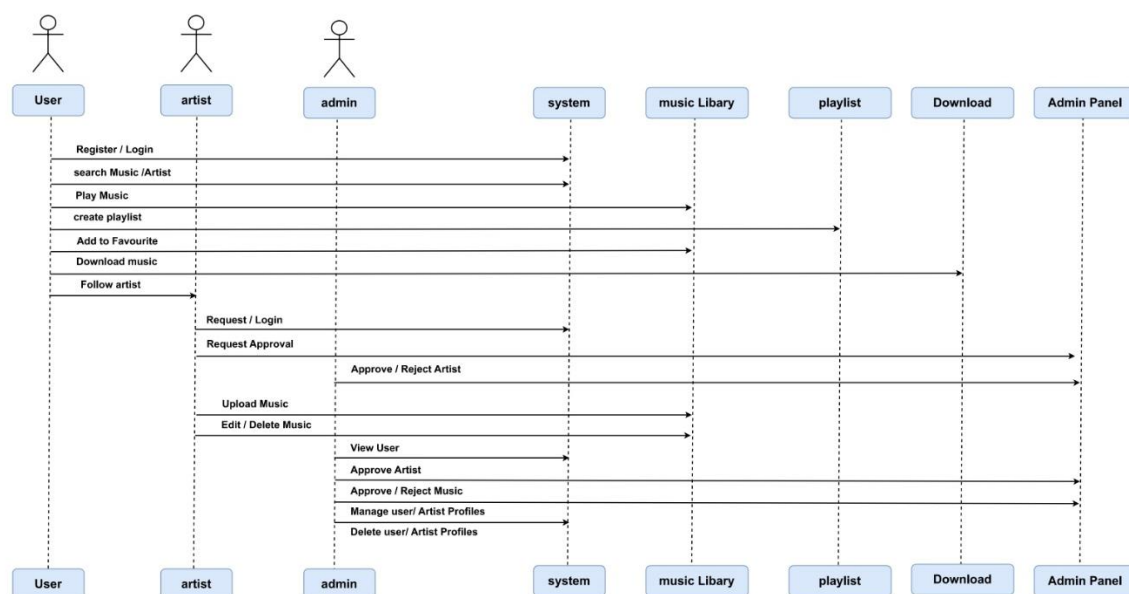
3.4.5 Interaction Diagram

An Interaction Diagram is a type of UML diagram that shows how objects in a system communicate with each other to perform a specific task. It focuses on the flow of messages between objects and the order in which these messages are exchanged. There are two main types of interaction diagrams: sequence diagrams and communication diagrams. Sequence diagrams emphasize the time sequence of messages, while communication diagrams highlight the relationships and interactions between objects. Interaction diagrams are useful for understanding system behaviour, visualizing workflows, and identifying how components collaborate to complete a process. They help developers design, analyze, and refine the logic of system interactions.



3.4.6 Sequence Diagram

A sequence diagram in UML visually depicts the interaction between objects or components over time, illustrating the flow of messages exchanged between them. It captures the dynamic behavior of a system by outlining the sequence of actions and their execution order, helping developers understand how different components interact in various scenarios. By providing clarity on message flow, object lifecycles, and control logic, sequence diagrams aid in system design, debugging, and communication, ensuring a well-structured and efficient software architecture.



3.5 MODULE DESIGN

The modular design of the Rental Platform System is structured to divide the platform into independent yet interconnected modules, each responsible for a specific set of functionalities. This approach promotes easier development, simplified maintenance, and scalability for future enhancements.

In the system, the **User Module** handles tenant and buyer functionalities, including **user registration, login, profile management, property browsing, search filtering, and initiating chat with landlords**. This module ensures a smooth and intuitive experience for users as they explore listings and communicate with property owners.

The **Landlord Module** empowers property owners or agents to **register, create property listings, update or delete posts, and chat with potential tenants or buyers**. It also supports profile management and provides a dashboard to track posted properties.

The **Admin Module** is responsible for **platform moderation**, including **reviewing property posts, approving or rejecting listings, and managing user accounts**. Admins maintain the overall integrity and security of the system.

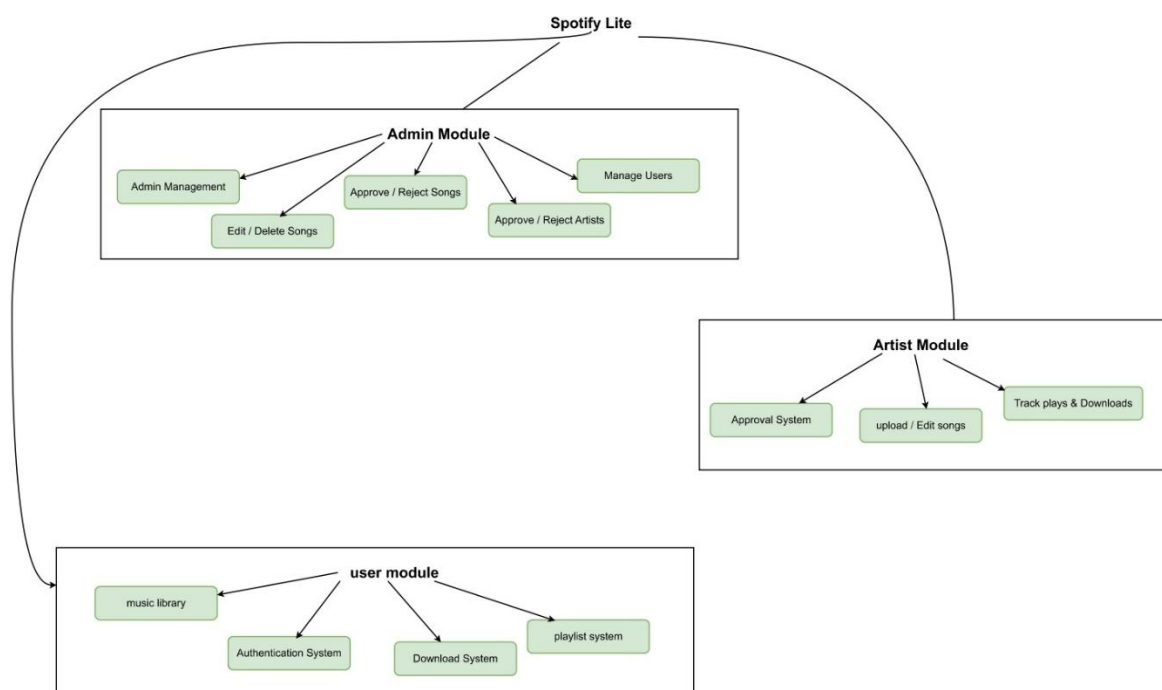
The **Frontend Module** is built using **ReactJS**, providing a responsive and interactive interface for both tenants/buyers and landlords. The UI is designed for clarity and usability across devices.

The **Backend Module** utilizes **Node.js and Express.js**, managing server-side logic, authentication, routing, and communication between the frontend and the database.

The **Database Module** is built on **MongoDB**, storing structured information such as user details, property listings, chat logs, and administrative records. It ensures high availability and secure access to data across modules.

This modular architecture allows for seamless updates, as changes in one module have minimal impact on others. It enhances collaborative development, simplifies debugging, and supports the integration of future features such as **location-based recommendations, automated alerts, or analytics dashboards** for landlords and admins.

3.5.1 Structure Chart



3.5.2 Modules Description

1. User Module

This module handles all functionalities for users who want to rent or buy properties. Users can register, log in, explore property listings, apply filters (location, price, type), save favorites, chat with landlords, and update their profiles. The interface is built using ReactJS, ensuring a seamless and responsive user experience across devices. This module focuses on easy navigation, real-time updates, and intuitive interactions.

2. Landlord Module

The landlord module enables property owners to register, log in, create listings, upload property images, and provide property details such as rental or selling price, type, and location. Landlords can manage existing posts (edit/delete) and communicate with users through the built-in chat feature. Built with ReactJS, this module offers a dashboard-like interface for smooth property and inquiry management.

3.6 INPUT DESIGN

The Input Design for the Rental Platform System emphasizes secure, validated, and user-friendly data entry across different modules. Structured inputs and validation mechanisms ensure smooth interaction for users and landlords while maintaining data consistency and integrity.

Module Inputs

1. User Interaction and Input

• User Registration and Login

- **Data Fields:** Full name, email, password, phone number.
- **Validation:**
 - Email format verification
 - Strong password enforcement
 - Phone number validation
 - Unique email check
- **Authentication:**
 - Encrypted inputs using JWT-based secure login mechanism.

• Property Search and Filter

- **Search Fields:** Location, property type (rent/buy), price range, number of bedrooms, etc.
- **Real-time Filters:**
 - Dropdowns, sliders, and input boxes for user-friendly filtering
- **Validation:**
 - Prevents empty or invalid values
 - Displays suggestions and matching results in real-time

- **Chat Interaction**

- **Inputs:** Text message to landlord
- **Validation:**
 - Input length control
 - Prevention of empty or abusive messages

- **Profile Update**

- **Fields:** Profile picture, name, contact info, preferences
- **Validation:**
 - Image file size/type
 - Non-empty fields check

2. Landlord Content Upload and Management

- **Landlord Registration and Login**

- **Data Fields:** Full name, email, password, phone number, profile image
- **Validation:**
 - Mandatory field enforcement
 - Image type/size check
 - Email uniqueness

- **Property Listing Creation**

- **Fields:** Property title, description, price, location, type (rent/sell), images
- **Validation:**
 - Image size/type (e.g., JPG, PNG)
 - Numeric input checks for price
 - Geolocation/address validation
 - Required field completion

- **Property Update/Delete**

- **Inputs:** Updated details (price, description, images), delete confirmation
- **Validation:**
 - Duplicate listing prevention
 - Image re-upload restrictions

- **Communication with Users**

- **Inputs:** Reply messages, contact sharing (if any)
- **Validation:**
 - Input sanitization
 - Character limits

3.7 OUTPUT DESIGN

The Output Design for the **Rental Platform System** focuses on providing accurate, intuitive, and visually appealing responses based on user interactions, landlord and tenant activities, and admin actions. The system ensures clear communication through responsive interfaces, real-time feedback, and well-structured data presentation for all stakeholders.

Module Outputs

1. User-Oriented Outputs

- **Property Listings Interface:**
 - **Real-Time Property Display:** Users see up-to-date listings with essential details such as property title, location, price, images, description, and landlord contact info. Interactive options like "Request a Visit," "Save to Favorites," and "Contact Landlord" are available.
 - **Search Results Display:** When users search for properties, results are displayed in a structured list with property images, prices, and other metadata. Filters such as location, price range, and property type help narrow down results.
- **Profile View (Tenant and Landlord):**
 - **Personalized Profile:** Tenants and landlords can view their profile, including past rental history, saved properties, and upcoming rental requests. For landlords, it also shows their listed properties, with options to edit or remove them.
- **Booking/Request Status:**
 - **Booking Confirmation:** After submitting a booking request, tenants receive confirmation messages like "Booking request submitted," along with further instructions or status updates.
 - **Request Updates:** Notifications for booking approvals, rejections, or cancellations, including reasons and next steps, keep tenants and landlords informed.
- **User Notifications:**
 - **Feedback Messages:** Messages like "Request submitted successfully," "No properties found," or "Profile updated" guide users throughout the platform, ensuring they are aware of the system's status and their actions.

2. Landlord-Oriented Outputs

- **Property Upload Confirmation:**
 - **Upload Feedback:** After uploading a property, landlords receive confirmation messages such as "Property listing submitted, awaiting approval."

- **Profile View (Landlord):**
 - **Property Management:** Landlords can view and manage their property listings, including details such as location, price, description, and images. Options to edit, delete, or mark properties as "rented" are available.
 - **Booking Request Notifications:**
 - **Request Alerts:** Landlords are notified when a tenant submits a booking request for their property, with details of the tenant and request status (approved, pending, or rejected).
 - **Content Status Reports:**
 - **Property Approval/Rejection:** Landlords are notified about the approval or rejection of their property listings, including feedback or reasons for rejection.
-

3. Admin-Oriented Outputs

- **Content Moderation Dashboard:**
 - **Pending Listings Overview:** Admins are provided with a dashboard to review pending property listings, with details like the landlord's name, property size, location, images, and submission date. Admins can approve or reject listings.
- **User and Property Management Reports:**
 - **User Account Overview:** Admins can view all registered users (landlords and tenants), their activity status, and flagged accounts for review. Admins also have access to reports on property listings and requests.
- **Analytics and System Logs:**
 - **Performance Reports:** Admins can access reports detailing system performance, user engagement trends (e.g., most popular locations, property types), and overall activity on the platform.
 - **Log Data:** Admins can review logs to troubleshoot issues or track user interactions across the system.

4. System-Wide Outputs

- **Responsive Web Pages:**
 - **Cross-Platform Compatibility:** All modules provide a responsive, user-friendly interface that adapts seamlessly to mobile and desktop views, ensuring consistent layout, fonts, and interactive elements.

SYSTEM REQUIREMENTS

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 INTRODUCTION

This project focuses on developing **Rental Platform System**, a web-based platform designed to connect landlords and tenants, enabling users to easily browse, rent, and manage properties. The system allows users to create, modify, and view property listings, while providing tenants with the ability to search for properties, book rental requests, and communicate with landlords. The platform is built using modern web technologies such as React for the front end, Node.js and Express for the back end, and MongoDB for database management.

The **Rental Platform System** offers features such as advanced property search with filter options, booking requests, user profiles for tenants and landlords, and a secure authentication system. Landlords can manage their property listings, track booking requests, and interact with tenants, while tenants can save properties, request visits, and view property details.

This platform aims to provide a seamless experience for both tenants and landlords by offering an intuitive interface and reliable data management, making the rental process efficient and transparent. The system is designed with scalability and performance in mind, ensuring smooth operation across all devices.

This project aims to create a user-friendly alternative to traditional rental platforms, fostering a transparent and community-driven environment for property renting and managing.

4.2 SOFTWARE SPECIFICATION

One of the most difficult task is selecting software for the system, once the system requirements is found out then we have to determine whether a particular software package fits for those system requirements. The application requirement:

Operating System	: Microsoft windows 8 or Above
Front End	: ReactJs, JavaScript, HTML, TailWind
Back End	: Node js, Express.js , Mongo Db
IDE	: VsCode ,Cursor

4.3 HARDWARE SPECIFICATION

The selection of hardware is very important in the existence and proper working of any software. Then selection hardware, the size and capacity requirements are also important.

Processor	: Pentium 4Processor/Above
Main Memory	: 4GB or Above
Hard Disk Capacity	: 40GB or Above
Mouse	: Any compatible
Keyboard	: Any compatible

4.4 TOOLS, PLATFORMS

This section describes the primary tools and platforms used in the development of the **Full Stack Rental Platform System**, covering both front-end and back-end technologies as well as the system environment.

4.4.1 Front-End Tools

The front end is responsible for the user interface, which is what the users see and interact with. The following technologies were used to develop the front-end:

- **React.js:** React.js is a popular JavaScript library for building user interfaces. It allows developers to create reusable UI components and provides a reactive data model. In the **Rental Platform System**, React.js is used to build a dynamic and responsive user interface. It enhances the user experience by ensuring smooth interactions and fast page loads.
- **HTML (HyperText Markup Language):** HTML is the foundational language for structuring web pages. It is used in the **Rental Platform System** to define the basic structure and content of the web pages. It helps in building the framework of the user interface that is displayed in web browsers.
- **Tailwind CSS:** Tailwind CSS is a utility-first CSS framework that provides pre-built classes to style the HTML elements. It allows for quick customization and development of responsive, attractive layouts without writing custom CSS from scratch. Tailwind is used in the **Rental Platform System** to create a modern, clean, and responsive design, making the platform look great on both mobile and desktop.
- **JavaScript (JS):** JavaScript is a client-side scripting language that adds interactivity to the web pages. It enables dynamic features like handling form submissions, creating real-time updates, and managing user actions. In the **Rental Platform System**, JavaScript is used to handle events like clicks, fetch data from the server, and manage user interactions with the platform.

4.4.2 Back-End Tools

The back end handles the server-side logic, data processing, and database management. The following tools are used for the back-end development of the **Rental Platform System**:

- **Node.js:** Node.js is a runtime environment built on Chrome's V8 JavaScript engine. It allows developers to run JavaScript code on the server side. In the **Rental Platform System**, Node.js is used to build the back-end logic and APIs, handling requests from the front-end and processing user actions such as property listings, registrations, and user interactions.
- **Express.js:** Express.js is a lightweight and flexible web application framework for Node.js. It simplifies routing, middleware management, and HTTP request handling. **Express.js** is used in the **Rental Platform System** to create RESTful APIs, manage server routes, and handle communication between the front-end and back-end.
- **MongoDB:** MongoDB is a NoSQL database that stores data in flexible, JSON-like documents. It is used in the **Rental Platform System** to store user profiles, property listings, and transaction data. MongoDB's flexibility makes it easy to handle various types of data without the need for predefined schemas, ensuring scalability and efficient data storage as the system grows.

4.4.3 Operating System and Development Environment

The application was developed on a personal device with the following configuration:

- **Device Name:** AnanthapadmanabhanHp
- **Processor:** 12th Gen Intel(R) Core(TM) i7-1260P @ 2.10 GHz
- **Installed RAM:** 16.0 GB
- **System Type:** 64-bit Operating System, x64-based Processor
- **Operating System:** Windows 11
- **Pen and Touch Support:** Not available

This setup provided sufficient processing power and memory for developing, testing, and deploying the application efficiently.

PROJECT IMPLEMENTATION

CHAPTER 5

PROJECT IMPLEMENTATION

5.1 INTRODUCTION

This chapter outlines the implementation phase of the **Full Stack Rental Platform System**, focusing on the key components of system development and integration. It covers the use of front-end and back-end frameworks, database setup, API development, and user interface design to ensure a seamless and interactive user experience.

The implementation process includes integrating core modules such as user authentication, property listing, search functionality, booking management, user profile management, and admin moderation. It also highlights the development of important features like property search, user account management, rental transactions, and property viewing.

The implementation phase incorporates code snippets for essential functions such as login validation, property listing management, user registration, and booking functionality. Special attention is given to code testing, debugging, and optimization to enhance the platform's performance, scalability, and maintainability.

To ensure the reliability of the system and user satisfaction, the implementation phase also involves unit testing, structured test planning, and validation procedures. These tests are designed to verify the accuracy of data transactions, responsiveness of the interface, and proper functioning of property listings, user interactions, and admin moderation features. The goal is to ensure that the **Full Stack Rental Platform System** operates efficiently and provides an engaging, user-friendly experience for both users and administrators.

5.2 CODING

5.2.1 CODING STANDARDS

In the development of the **Full Stack Rental Platform System**, strict adherence to standardized coding practices ensures that the application is readable, maintainable, and scalable. The front-end of the platform, built using **ReactJS**, **HTML**, **Tailwind CSS**, and **JavaScript**, follows consistent indentation and formatting across all files to improve readability. Meaningful variable naming is enforced throughout, with descriptive names for components, functions, and variables to enhance code clarity. React components are modularized to ensure reusability and maintainability, with a clear separation of concerns between the logic and UI layers. **State management** in React is handled through the **Context API**, allowing for centralized state management, while **Tailwind CSS** ensures rapid styling with utility-first classes, maintaining a responsive and consistent design across the application.

On the back-end, **Node.js** and **Express.js** are used to build the server-side logic, following best practices for JavaScript development. Functions and variables are named descriptively, ensuring the code is easy to understand and maintain. The back-end is structured into modular services for different features, such as user authentication, property management, and booking, with each service having its own controller, model, and route files. This modular approach simplifies both development and future scalability. Robust **error handling** and input **validation** are implemented to ensure data integrity and smooth user interactions. The API is designed using **RESTful principles**, with clear, logically named endpoints like `/createProperty` or `/getUserProfile`, reflecting the specific operations they perform.

Version control is managed using **Git**, with clear commit messages and a branching strategy to streamline the development process. The codebase is well-commented to explain complex logic and algorithms, ensuring that it is easy for future developers to understand and build upon. These practices collectively contribute to a well-organized, stable, and maintainable codebase, promoting smooth collaboration and future enhancements for the **Full Stack Rental Platform System**.

5.2.2 SAMPLE CODE

```
auth.controller.js

import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";
import prisma from "../lib/prisma.js";

export const register = async (req, res) => {
  const {username, email, password} = req.body;

  try{

    // hash the password

    const hashedPassword = await bcrypt.hash(password, 10);

    console.log(hashedPassword)
```

```
// Creating a new user

const newUser = await prisma.user.create({
  data: {
    username,
    email,
    password: hashedPassword,
  }
});

console.log(newUser);

res.status(201).json({message:"User created successfully"})
} catch(err){
  console.log(err)
  res.status(500).json({message:"Failed to create user"})
}

};

export const login = async (req,res) => {

  const {username,password} = req.body;
  try {
    // check the user exists
    const user = await prisma.user.findUnique({
      where: {username}
    })

    if(!user){
      return res.status(401).json({message:"Invalid Credentials!"});
    }
  }
}
```

```
}

// check if the passowrd is correct

const isPasswordValid = await bcrypt.compare(password, user.password);

if (!isPasswordValid) {
  return res.status(401).json({message:"Invalid Credentials!"});
}

// generate cookie token and send to user

// res.setHeader("Set-Cookie", "test=" + "myValue").json("Success")
const age = 1000 * 60 * 60 * 24 * 7;

const token = jwt.sign({
  id: user.id,
  isAdmin : false,
}, process.env.JWT_SECRET_KEY,
{expiresIn: age}
);

const {password: userPassword, ...userInfo} = user;

res.cookie("token", token, {
  httpOnly:true,
  // secure:true,
  maxAge: age,
}).status(200).json(userInfo)
} catch (error) {
```

```
    console.log(error.message);
    res.status(500).json({message:"Failed to login!"})
  }

}

export const logout = (req,res) => {
  res.clearCookie("token").status(200).json({message:"Logout Successful"})
}
```

chat.controller.js

```
import prisma from "../lib/prisma.js";

export const getChats = async (req, res) => {

  const tokenUserId = req.userId;

  try {
    const chats = await prisma.chat.findMany({
      where: {
        userIDs: {
          hasSome: [tokenUserId],
        }
      }
    });

    for(const chat of chats) {
      const receiverId = chat.userIDs.find(id => id !== tokenUserId);

      const receiver = await prisma.user.findUnique({
        where: {
          id: receiverId,
```

```
    },
    select: {
      id: true,
      username: true,
      avatar: true,
    }
  });
  chat.receiver = receiver;
}
res.status(200).json(chats);
} catch (err) {
  console.log(err);
  res.status(500).json({ message: "Failed to get chats!" });
}
}
```

```
export const findOrCreateChat = async (req, res) => {
  const tokenUserId = req.userId;
  const { receiverId } = req.body;

  try {
    // Modified query to ensure both users are in the chat
    const existingChat = await prisma.chat.findFirst({
      where: {
        AND: [
          {
            userIDs: {
              has: tokenUserId
            }
          },
        ],
      },
    });
```

```
        {
          userIDs: {
            has: receiverId
          }
        }
      ]
    }
  });

  if (existingChat) {
    return res.status(200).json(existingChat);
  }

  // If no chat exists, create a new one
  const newChat = await prisma.chat.create({
    data: {
      userIDs: [tokenUserId, receiverId],
    }
  });

  // Update both users' chatIDs
  await prisma.user.update({
    where: { id: tokenUserId },
    data: {
      chatIDs: {
        push: newChat.id
      }
    }
  });
});
```

```
    await prisma.user.update({
      where: { id: receiverId },
      data: {
        chatIDs: {
          push: newChat.id
        }
      }
    });

    res.status(200).json(newChat);
  } catch (err) {
    console.log(err);
    res.status(500).json({ message: "Failed to find or create chat!" });
  }
}

export const getChat = async (req, res) => {
  const tokenUserId = req.userId;
  const chatId = req.params.id;

  console.log("Getting chat with ID:", chatId);
  console.log("For user ID:", tokenUserId);

  try {
    // First get the chat
    const chat = await prisma.chat.findUnique({
      where: {
        id: chatId,
        userIDs: {
          hasSome: [tokenUserId],

```

```
    },  
  },  
  include: {  
    messages: {  
      orderBy: {  
        createdAt: "asc",  
      },  
    },  
  },  
});  
  
console.log("Found chat:", chat);  
  
if (!chat) {  
  console.log("Chat not found!");  
  return res.status(404).json({ message: "Chat not found!" });  
}  
  
// Get receiver info  
const receiverId = chat.userIDs.find(id => id !== tokenUserId);  
const receiver = await prisma.user.findUnique({  
  where: { id: receiverId },  
  select: {  
    id: true,  
    username: true,  
    avatar: true,  
  }  
});  
  
console.log("Found receiver:", receiver);
```

```
// Update seenBy only if not already seen
if (!chat.seenBy.includes(tokenUserId)) {
  await prisma.chat.update({
    where: { id: chatId },
    data: {
      seenBy: {
        push: tokenUserId,
      },
    },
  });
}

// Add receiver info to the chat object
const chatWithReceiver = {
  ...chat,
  receiver: receiver
};

console.log("Returning chat with receiver:", chatWithReceiver);
res.status(200).json(chatWithReceiver);
} catch (err) {
  console.error("Error in getChat:", err);
  res.status(500).json({ message: "Failed to get chat!" });
}
}

export const addChat = async (req, res) => {

  const tokenUserId = req.userId;
```

```
try {
  const newChat = await prisma.chat.create({
    data: {
      userIDs: [tokenUserId, req.body.receiverId],
    }
  })
  res.status(200).json(newChat);
} catch (err) {
  console.log(err);
  res.status(500).json({ message: "Failed to add chat!" });
}

export const readChat = async (req, res) => {
  const tokenUserId = req.userId;

  try {
    const chat = await prisma.chat.findUnique({
      where: {
        id: req.params.id,
        userIDs: {
          hasSome: [tokenUserId],
        },
      },
    });

    if (!chat) {
      return res.status(404).json({ message: "Chat not found!" });
    }
  }
```

```
// Only update if not already seen
if(!chat.seenBy.includes(tokenUserId)) {
  const updatedChat = await prisma.chat.update({
    where: { id: req.params.id },
    data: {
      seenBy: {
        push: tokenUserId,
      },
    },
  });
  return res.status(200).json(updatedChat);
}

res.status(200).json(chat);
} catch (err) {
  console.log(err);
  res.status(500).json({ message: "Failed to read chat!" });
}
}
```

message.controller.js

```
import prisma from "../lib/prisma.js";

export const addMessage = async (req, res) => {
  const tokenUserId = req.userId;
  const chatId = req.params.chatId;
  const text = req.body.text;

  // Validate message
  if (!text || typeof text !== 'string' || text.trim().length === 0) {
```

```
    return res.status(400).json({ message: "Message text is required!" });
  }

  if (text.length > 1000) {
    return res.status(400).json({ message: "Message too long (max 1000 chars)!" });
  }

  try {
    const chat = await prisma.chat.findUnique({
      where: {
        id: chatId,
        userIDs: {
          hasSome: [tokenUserId],
        },
      },
    });

    if (!chat) {
      return res.status(404).json({ message: "Chat not found!" });
    }

    const message = await prisma.message.create({
      data: {
        text: text.trim(),
        chatId,
        userId: tokenUserId,
      },
    });

    // Update chat with new message and seen status
```

```
await prisma.chat.update({
  where: { id: chatId },
  data: {
    lastMessage: text.trim(),
    seenBy: {
      set: [tokenUserId], // Only mark as seen by sender
    },
  },
});

res.status(200).json(message);
} catch (err) {
  console.log(err);
  res.status(500).json({ message: "Failed to add message!" });
}
}
```

post.controller.js

```
import prisma from "../lib/prisma.js";

export const getPosts = async (req, res) => {
  const query = req.query;

  const filters = {
    where: {
      city: query.city || undefined,
      type: query.type || undefined,
      property: query.property || undefined,
      bedroom: query.bedroom ? parseInt(query.bedroom) : undefined,
      price: {
```

```
      gte: query.minPrice ? parseInt(query.minPrice) : 0,
      lte: query.maxPrice ? parseInt(query.maxPrice) : 10000000,
    },
  },
};

try {
  const posts = await prisma.post.findMany(filters);
  res.status(200).json(posts);
} catch (err) {
  console.error("Error fetching posts:", err);
  res.status(500).json({ message: "Failed to get posts" });
}
};

export const getPost = async (req, res) => {
  const id = req.params.id;
  const tokenUserId = req.userId; // Get the user ID from the token

  try {
    const post = await prisma.post.findUnique({
      where: { id },
      include: {
        postDetail: true,
        user: {
          select: {
            username: true,
            avatar: true,
          }
        }
      }
    })
  }
```

```
    }  
  });  
  
  // Check if post exists  
  if (!post) {  
    return res.status(404).json({ message: "Post not found" });  
  }  
  
  // Check if the user has saved this post  
  let isSaved = false;  
  if (tokenUserId) {  
    const savedPost = await prisma.savePost.findUnique({  
      where: {  
        userId_postId: {  
          userId: tokenUserId,  
          postId: id,  
        },  
      },  
    });  
    isSaved = !!savedPost;  
  }  
  
  res.status(200).json({  
    ...post,  
    isSaved // Add the isSaved flag to the response  
  });  
  // console.log("Post fetched successfully:", post);  
  // console.log(isSaved);
```

```
    } catch (err) {  
      console.log(err);  
      res.status(500).json({ message: "Error getting post" });  
    }  
  }  
}
```

```
export const addPost = async (req, res) => {  
  const body = req.body;  
  const tokenUserId = req.userId;  
  try {  
    const newPost = await prisma.post.create({  
      data: {  
        ...body.postData,  
        userId: tokenUserId,  
        postDetail: {  
          create: body.postDetail,  
        },  
      },  
    });  
    res.status(200).json(newPost);  
  } catch (err) {  
    console.log(err);  
    res.status(500).json({ message: "Error creating post" });  
  }  
}
```

```
export const updatePost = async (req, res) => {  
  const id = req.params.id;  
  const tokenUserId = req.userId;  
  const { postData, postDetail } = req.body;
```

```
try {  
  const existingPost = await prisma.post.findUnique({  
    where: { id }  
  });  
  
  if (!existingPost) {  
    return res.status(404).json({ message: "Post not found" });  
  }  
  
  if (existingPost.userId !== tokenUserId) {  
    return res.status(403).json({ message: "You are not authorized to update this post" });  
  }  
  
  const updatedPost = await prisma.post.update({  
    where: { id },  
    data: {  
      ...postData,  
      postDetail: {  
        update: postDetail  
      }  
    }  
  });  
  
  res.status(200).json(updatedPost);  
} catch (err) {  
  console.log(err);  
  res.status(500).json({ message: "Error updating post" });  
}  
};
```

```
export const deletePost = async (req, res) => {  
  const id = req.params.id;  
  const tokenUserId = req.userId;  
  
  try {  
    const post = await prisma.post.findUnique({  
      where: { id },  
      include: {  
        postDetail: true,  
        savedPost: true, // optional if you want to check before deletion  
      },  
    });  
  
    if (!post) {  
      return res.status(404).json({ message: "Post not found" });  
    }  
  
    if (post.userId !== tokenUserId) {  
      return res.status(403).json({ message: "You are not authorized to delete this post" });  
    }  
  
    // 1. Delete related PostDetail  
    if (post.postDetail) {  
      await prisma.postDetail.delete({  
        where: { PostId: id },  
      });  
    }  
  
    // 2. Delete related SavedPosts
```

```
await prisma.savedPost.deleteMany({
  where: { postId: id },
});
```

```
// 3. Delete Post
```

```
await prisma.post.delete({
  where: { id },
});
```

```
res.status(200).json({ message: "Post deleted successfully" });
} catch (err) {
  console.log(err);
  res.status(500).json({ message: "Error deleting post" });
}
};
```

test.controller.js

```
import jwt from "jsonwebtoken";
```

```
export const shouldBeLoggedIn = async (req, res) => {
  console.log(req.userId);

  res.status(200).json({ message: 'You are Authenticated' });
}

export const shouldBeAdmin = async (req, res) => {
  const token = req.cookies.token;

  if (!token) {
    return res.status(401).json({ message: 'You are not logged in' });
  }
}
```

```
jwt.verify(token, process.env.JWT_SECRET_KEY, async (err, payload) => {  
  if (err) {  
    return res.status(403).json({ message: 'Invalid token' });  
  }  
  
  if (!payload.isAdmin) {  
    return res.status(403).json({ message: 'You are not an admin' });  
  }  
  
  res.status(200).json({ message: 'You are Authenticated' });  
});  
}
```

user.controller.js

```
import prisma from "../lib/prisma.js";  
import bcrypt from "bcrypt";  
  
export const getUsers = async (req, res) => {  
  console.log("It works!");  
  
  try {  
    const users = await prisma.user.findMany();  
    res.status(200).json(users);  
  } catch (err) {  
    console.log(err);  
    res.status(500).json({ message: "Failed to get users!" });  
  }  
}  
  
export const getUser = async (req, res) => {
```

```
const id = req.params.id;

try {
  const user = await prisma.user.findUnique({
    where: { id },
  });
  res.status(200).json(user);

} catch (err) {
  console.log(err);
  res.status(500).json({ message: "Failed to get user!" });
}

export const updateUser = async (req, res) => {
  const id = req.params.id;
  const tokenUserId = req.userId;
  const {password, avatar, ...inputs} = req.body;

  if (id !== tokenUserId) {
    return res.status(403).json({ message: "Not authorized" });
  }

  let updatedPassword = null;
  try {
    if (password) {
      updatedPassword = await bcrypt.hash(password, 10);
    }
  }
```

```
const updatedUser = await prisma.user.update({
  where: { id },
  data: {
    ...inputs,
    ...(updatedPassword && { password: updatedPassword }),
    ...(avatar && { avatar }),
  }
});

const { password:userPassword, ...rest } = updatedUser;
res.status(200).json(rest);

} catch (err) {
  console.log(err);
  res.status(500).json({ message: "Failed to update user!" });
}
}

export const deleteUser = async (req, res) => {

  const id = req.params.id;
  const tokenUserId = req.userId;

  if (id !== tokenUserId) {
    return res.status(403).json({ message: "Not authorized" });
  }

  try {
    await prisma.user.delete({
      where: { id },
    });
  }
}
```

```
    res.status(200).json({ message: "User has been deleted" });
  } catch (err) {
    console.log(err);
    res.status(500).json({ message: "Failed to delete user!" });
  }
}

export const savePost = async (req, res) => {
  const { postId } = req.body;
  const tokenUserId = req.userId;

  try {
    // First verify the post exists
    const postExists = await prisma.post.findUnique({
      where: { id: postId }
    });

    if (!postExists) {
      return res.status(404).json({ message: "Post not found" });
    }

    // Check if already saved - using findFirst for MongoDB
    const savedPost = await prisma.savedPost.findFirst({
      where: {
        userId: tokenUserId,
        postId: postId
      }
    });

    if (savedPost) {
      await prisma.savedPost.delete({
```

```
        where: {
          id: savedPost.id
        }
      });

      return res.status(200).json({ message: "Post unsaved successfully" });
    } else {
      await prisma.savedPost.create({
        data: {
          userId: tokenUserId,
          postId: postId
        }
      });

      return res.status(200).json({ message: "Post saved successfully" });
    }
  } catch (err) {
    console.log(err);
    res.status(500).json({ message: "Failed to save post" });
  }
}
```

// In user.controller.js

```
export const profilePosts = async (req, res) => {
  const tokenUserId = req.userId; // Changed from req.params.userId to req.userId
  try {
    const userPosts = await prisma.post.findMany({
      where: { userId: tokenUserId },
    });

    const saved = await prisma.savedPost.findMany({ // Changed from savePost to savedPost
      where: { userId: tokenUserId },
      include: { post: true },
    });
```

```
    });

    const savedPosts = saved.map((item) => item.post);
    res.status(200).json({userPosts, savedPosts});

  } catch (err) {
    console.log(err);
    res.status(500).json({ message: "Failed to get profilePosts!" });
  }
}

export const getNotificationNumber = async (req, res) => {
  const tokenUserId = req.userId;
  try {
    const number = await prisma.chat.count({
      where: {
        userIDs: {
          hasSome: [tokenUserId],
        },
        NOT: {
          seenBy: {
            hasSome: [tokenUserId], // Wrap in array
          },
        }
      }
    });
  });

  res.status(200).json(number);

  } catch (err) {
```

```
        console.log(err);
        res.status(500).json({ message: "Failed to get notification count!" });
    }
}
```

verifyToken.js

```
import jwt from "jsonwebtoken";

export const verifyToken = async (req, res, next) => {
    const token = req.cookies.token;

    if (!token) {
        return res.status(401).json({ message: 'You are not logged in' });
    }

    jwt.verify(token, process.env.JWT_SECRET_KEY, async (err, payload) => {
        if (err) {
            return res.status(403).json({ message: 'Invalid token' });
        }
        req.userId = payload.id;
        next();
    });
}
```

HomePage.jsx

```
import React, { useContext } from 'react';
import SearchBar from '../components/SearchBar';
import { AuthContext } from '../context/AuthContext';
import { Link } from 'react-router-dom';

const HomePage = () => {
```

```
const { currentUser } = useContext(AuthContext);

const stats = [
  { value: '10,000+', label: 'Properties Listed' },
  { value: '5,000+', label: 'Happy Customers' },
  { value: '50+', label: 'Cities Covered' },
  { value: '24/7', label: 'Customer Support' },
];

return (
  <div className="min-h-screen bg-white">
    {/* Hero Section - Adjusted padding to move content up */}
    <div className="relative overflow-hidden bg-gray-900 pb-24 sm:pb-32 lg:pb-40">
      <div className="absolute inset-0">
        
      </div>
      {/* Significantly reduced top padding here */}
      <div className="relative pt-24 pb-24 sm:pt-16 sm:pb-20 lg:pt-20 lg:pb-28">
        <div className="mx-auto max-w-7xl px-6 lg:px-8">
          <div className="mx-auto max-w-2xl text-center">
            <h1 className="text-4xl font-bold tracking-tight text-white sm:text-5xl lg:text-6xl">
              Find Your <span className="text-indigo-400">Dream Home</span> with Ease
            </h1>
            <p className="mt-4 text-xl leading-8 text-indigo-100">
              Propease connects you with the perfect property tailored to your lifestyle and budget.
            </p>
          </div>
        </div>
      </div>
    </div>
  </div>
)
```

```

    </p>

    <div className="mt-8 flex items-center justify-center">

      <SearchBar />

    </div>

  </div>

</div>

</div>

</div>

{ /* Rest of the code remains the same */ }

{ /* Stats Section */ }

<div className="bg-white py-24 sm:py-32">

  <div className="mx-auto max-w-7xl px-6 lg:px-8">

    <div className="mx-auto max-w-2xl lg:max-w-none">

      <div className="text-center">

        <h2 className="text-3xl font-bold tracking-tight text-gray-900 sm:text-4xl">

          Trusted by homeowners and investors

        </h2>

        <p className="mt-4 text-lg leading-8 text-gray-600">

          Join thousands who've found their perfect property through Propease

        </p>

      </div>

      <div className="mt-16 grid grid-cols-1 gap-0.5 overflow-hidden rounded-2xl text-center sm:grid-cols-2 lg:grid-cols-4">

        {stats.map((stat, statIdx) => (

          <div key={statIdx} className="flex flex-col bg-indigo-50 p-8">

            <dt className="text-sm font-semibold leading-6 text-indigo-600">{stat.label}</dt>

            <dd className="order-first text-3xl font-semibold tracking-tight text-indigo-900">

              {stat.value}

            </dd>

          </div>

        ))}

      </div>

    </div>

  </div>

</div>

```

```
</div>

))}
</div>

</div>

</div>

</div>

{ /* CTA Section */ }
<div className="relative bg-indigo-700">
  <div className="absolute inset-0">
    
  </div>
  <div className="relative mx-auto max-w-7xl py-24 px-6 sm:py-32 lg:px-8">
    <div className="text-center">
      <h2 className="text-3xl font-bold tracking-tight text-white sm:text-4xl">
        Ready to find your dream home?
      </h2>
      <p className="mx-auto mt-6 max-w-2xl text-lg leading-8 text-indigo-200">
        Our expert agents are ready to help you every step of the way.
      </p>
      <div className="mt-10 flex items-center justify-center gap-x-6">
        {currentUser ? (
          <Link
            to="/list"
```

```
        className="rounded-md bg-white px-6 py-3 text-sm font-semibold text-indigo-600 shadow-sm hover:bg-indigo-50 focus-visible:outline focus-visible:outline-offset-2 focus-visible:outline-white"
```

```
    >
```

```
        Browse Properties
```

```
    </Link>
```

```
  ) : (
```

```
    <
```

```
    <Link
```

```
      to="/register"
```

```
        className="rounded-md bg-indigo-500 px-6 py-3 text-sm font-semibold text-white shadow-sm hover:bg-indigo-400 focus-visible:outline focus-visible:outline-offset-2 focus-visible:outline-indigo-500"
```

```
    >
```

```
        Get Started
```

```
    </Link>
```

```
  </>
```

```
  )}
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
);
```

```
};
```

```
export default HomePage;
```

App.jsx

```
import React from 'react';
```

```
import {
```

```
  createBrowserRouter,
```

```
  RouterProvider,
```

```
} from 'react-router-dom';
import HomePage from './routes/HomePage';
import ListPage from './routes/ListPage';
import SinglePage from './routes/SinglePage';
import ProfilePage from './routes/ProfilePage';
import ProfileUpdatePage from './routes/ProfileUpdatePage';
import Register from './routes/Register';
import Login from './routes/Login';
import { Layout, RequireAuth } from './routes/Layout';
import NewPostPage from './routes/NewPostPage';
import { listPageLoader, profilePageLoader, singlePageLoader } from './lib/loaders';
const router = createBrowserRouter([
  {
    element: <Layout />, // Public layout wrapper
    children: [
      { index: true, element: <HomePage /> },
      { path: 'list', id: 'posts', element: <ListPage/> , loader: listPageLoader },
      { path: 'login', element: <Login /> },
      { path: 'register', element: <Register /> },
    ],
  },
  {
    element: <RequireAuth />, // Protected routes wrapper
    children: [
      { path: 'profile', element: <ProfilePage />, loader: profilePageLoader},
      { path: 'profile/c/:chatId', element: <ProfilePage />, loader: profilePageLoader},
      { path: 'profile/update', element: <ProfileUpdatePage /> },
      { path: ':id', element: <SinglePage />, loader: singlePageLoader},
      { path: 'add', element: <NewPostPage /> },
    ],
  },
],
```

```
    },  
  ]);  
  function App() {  
    return <RouterProvider router={router} />;  
  }  
  export default App;
```

Login.jsx

```
import { useContext, useState } from "react";  
import { Link, useNavigate } from "react-router-dom";  
import apiRequest from "../lib/apiRequest";  
import { AuthContext } from "../context/AuthContext";  
  
function Login() {  
  const [error, setError] = useState("");  
  const [isLoading, setIsLoading] = useState(false);  
  
  const {updateUser} = useContext(AuthContext);  
  
  const navigate = useNavigate();  
  
  const handleSubmit = async (e) => {  
    e.preventDefault();  
    setIsLoading(true);  
    setError("");  
    const formData = new FormData(e.target);  
  
    const username = formData.get("username");  
    const password = formData.get("password");
```

```
try {  
  const res = await apiRequest.post("/auth/login", {  
    username,  
    password,  
  });  
  
  updateUser(res.data);  
  
  navigate("/");  
}  
catch (err) {  
  console.log(err);  
  setError(err.response.data.message);  
} finally {  
  setIsLoading(false);  
}  
};  
  
return (  
  <div className="flex min-h-screen w-full">  
    <div className="flex items-center justify-center w-full md:w-1/2 px-6 py-8 mb-32">  
      <div className="w-full max-w-md">  
        <form onSubmit={handleSubmit} className="flex flex-col space-y-6">  
          <h1 className="text-3xl font-bold text-gray-800 mb-2">Welcome back</h1>  
          <input  
            name="username"  
            required  
            minLength={3}  
            maxLength={20}
```

```
        type="text"
        placeholder="Username"
        className="px-4 py-3 border border-gray-300 rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500"
    />
    <input
        name="password"
        type="password"
        required
        placeholder="Password"
        className="px-4 py-3 border border-gray-300 rounded-lg focus:outline-none
focus:ring-2 focus:ring-blue-500"
    />
    <button disabled={isLoading}
        className="py-3 px-4 bg-blue-600 text-white font-medium rounded-lg hover:bg-
blue-700 transition duration-200 disabled:opacity-50 disabled:cursor-not-allowed"
    >
        Login
    </button>

    {error && <span className="text-red-500 text-sm">{error}</span>}

    <Link to="/register" className="text-blue-600 hover:text-blue-800 text-sm text-
center">
        Don't you have an account?
    </Link>
</form>
</div>
</div>
<div className="hidden md:block md:w-1/2 bg-gray-100">
    
```

```
</div>

</div>

);
}
```

export default Login;

5.2.3 Code Validation and Optimization

5.2.3.1 Code Validation

Code validation in the **Full Stack Rental Platform System** ensures smooth functionality and adherence to industry standards, enhancing reliability and maintainability. The validation process involved the following steps:

- **Syntax Checking:** Front-end and back-end code were regularly checked using **ESLint** for JavaScript and **Prettier** for consistent formatting across the project.
- **Logical Error Detection:** Unit tests and manual testing were conducted to verify modules like user authentication, property listings, booking management, and payments worked as expected. This helped identify and resolve logic errors early.
- **Adherence to Standards:** Coding practices across **ReactJS**, **Tailwind CSS**, **Node.js**, **Express.js**, and **MongoDB** were aligned with industry best practices, ensuring clean, modular, and maintainable code.

3.2.3.2 Code Optimization

To ensure the **Full Stack Rental Platform System** runs efficiently and can scale with growing users and data, several code optimization strategies were implemented:

- **Database Query Optimization:** Redundant queries were minimized, and only the necessary fields were selected during database queries to reduce load time and optimize performance.
- **Caching Mechanisms:** Frequently accessed data, like property listings and user details, were cached using in-memory storage methods to reduce backend requests and speed up page load times.
- **Code Refactoring:** Repetitive and complex logic was refactored into reusable components and functions. Modular design principles were applied across both front-end and back-end to enhance code readability, speed, and maintainability. These optimizations ensured that the platform performs well and is capable of handling increased traffic and data over time.

5.3 DEBUGGING

Debugging is a crucial part of the **Full Stack Rental Platform System** development process to ensure smooth operation across both the front-end and back-end systems. Given the platform's focus on property listings, user interaction, and secure transactions, any interruption could negatively affect user experience. This section outlines the structured debugging approach used to maintain functionality, reliability, and performance across web and mobile interfaces.

1. Front-End Debugging

1.1 User Interface and Layout Issues

The front-end, developed using ReactJS and TailwindCSS, requires consistent styling and layout across devices.

- **Cross-platform inconsistencies** were debugged using browser dev tools and React DevTools.
- **Component misalignment and layout issues** were resolved by inspecting the DOM structure and applying proper styling with TailwindCSS classes.
- **Network errors during API calls** were tracked using browser's Network tab and React's console logs.

1.2 Interaction and State Management

Interactive features such as form submissions, user authentication, and dynamic page content can sometimes cause issues.

- **State management problems** were resolved using React's state management features like Context API and Redux.
- **Performance issues in rendering** were debugged using React's Profiler tool and optimized by using memoization techniques (e.g., useMemo, React.memo).

2. Back-End Debugging

2.1 API and Data Handling Issues

The back-end, built using Node.js and Express.js, serves RESTful APIs for property listings, user authentication, and transaction management.

- **Faulty API calls** were traced using the built-in logging middleware in Express.js.
- **Postman and Swagger** were used to simulate API requests and validate responses from the back-end.
- **Authentication and data flow issues** were traced through endpoint-level logs and tested for edge cases like invalid user credentials or missing data.

2.2 Database Debugging

MongoDB was used to store user profiles, property data, and transaction history.

- **Database query issues** were debugged using MongoDB Atlas logs and performance analysis tools.
- **Slow queries and data retrieval issues** were resolved by optimizing MongoDB queries and indexing frequently accessed fields.

3. Network & Performance Debugging

3.1 Real-Time Issues

The platform uses cloud-based storage and third-party APIs to handle property data and images.

- **Slow data retrieval** was debugged by analyzing network latency and testing data delivery under different conditions.
- **API integration issues** were detected and resolved by inspecting API response times and error messages.

3.2 Authentication and Session Management

Secure login and user authentication are crucial for a personalized experience.

- **JWT token expiration issues** were traced through logs and analyzed to ensure proper session handling and token refreshing.

4. Logging and Error Tracking

Structured logging was implemented using **Winston** for back-end logging and browser console logs for front-end error tracking.

- Logs captured failed logins, API call failures, and property-related errors.
- Logs were periodically reviewed to detect recurring bugs and performance issues.

5. Real-Time Debugging & Recovery

5.1 Handling App Crashes and UI Freezes

- **Error boundaries in React** were used to prevent app-wide crashes and manage errors in specific components.
- **Crash reports and stack traces** were captured using tools like Sentry and React Error Boundaries.

5.2 Auto Recovery & Error Messages

- **Retry mechanisms** were implemented for failed API calls and broken image links.
- Users received **meaningful error messages** (e.g., “Failed to load property details”) to improve troubleshooting and reduce user frustration.

5.4 UNIT TESTING

Unit testing, test planning, and test case development are essential in ensuring the **Full Stack Rental Platform System** delivers a smooth, accurate, and reliable user experience. The platform handles various interactions, such as user authentication, property listings, transaction management, and content updates. Rigorous testing is required to validate these features and maintain high system performance and usability.

5.4.1 Unit Testing

Unit testing involves testing individual modules—such as user registration, property listing, and transaction management—independently to ensure they function as expected. This helps detect bugs early in the development process.

Purpose:

- Validate the **user authentication** module to ensure secure login, registration, and session management.
- Ensure **property listing** and searching work without errors or delays.
- Test **transaction management**, ensuring rent or purchase actions are processed correctly.
- Confirm **admin functionality**, including property approval, rejection, and user management, operates as intended.
- Identify **API communication errors** between the front-end (ReactJS) and back-end (Node.js/MongoDB).

5.4.2 Test Plan

Objectives:

- Ensure each functional module—**User**, **Admin**, **Property**, **Transaction**, and **Search**—functions correctly under both normal and edge cases.
- Validate the **listing functionality** by testing adding, editing, and deleting property listings.
- Test the **transaction flow** to ensure rent and purchase actions are properly managed.
- Verify **system behavior** under failure conditions (e.g., no internet, failed property uploads, invalid credentials).

Testing Environment:

- **Front-End:** ReactJS (Web)
- **Back-End:** Node.js (Express), MongoDB
- **Development Tools:** Visual Studio Code, Postman, React DevTools, Browser Dev Console
- **Testing Tools:** Jest (for React), Mocha/Chai (for Node.js), Postman (for API testing)

- **Test Scope:**

- **Functional Testing:** Test all user scenarios such as login, property listing, searching, and transaction management.
- **Performance Testing:** Assess the load time for property listings and responsiveness of user interactions.
- **UI Testing:** Ensure responsive design and visual consistency across devices.
- **Error Handling Testing:** Test edge cases like invalid user data, failed property uploads, broken links, and system downtime.

5.4.3 Test Cases

Test Case	Description	Expected Result
Login with valid credentials	User logs in with correct email and password	Redirect to home page
Login with invalid credentials	User logs in with incorrect credentials	Display error message “Invalid credentials”
Add a new property listing	User adds a new property with details	Property appears in user’s dashboard
Search for a property	User enters a property name or filter in search bar	Relevant property listings are displayed
Create a new tenant transaction	User initiates a transaction for rent or purchase	Transaction is recorded and confirmed
Upload property image as landlord	Landlord uploads property images with details	Property images are sent for admin approval
Admin approves property listing	Admin reviews and approves the uploaded property listing	Property becomes visible to users in search results
View property details	User clicks on a property listing to view details	Property details (address, price, description) are displayed
System without internet	Attempt to load property listing while offline	Display message “No internet connection”

SYSTEM TESTING

CHAPTER 6

SYSTEM TESTING

6.1 INTRODUCTION

System testing is a crucial phase in the development of the rental platform system, ensuring that the entire application functions correctly across different real-world scenarios and user interactions. The objective of this phase is to validate both the front-end and back-end components, ensuring functionality, reliability, performance, and user experience before deployment.

The platform integrates key elements such as a React-based user interface, a Node.js back-end, and MySQL for data storage. It offers features like property listing, tenant and landlord interactions, transaction management, and admin moderation. As such, comprehensive system testing is essential to ensure that all modules work harmoniously and that the system meets its specified requirements.

During system testing, both integration testing and end-to-end functional testing are performed. Integration testing ensures smooth interactions between components like user authentication, property search, and database transactions. Full system testing verifies behavior across various use cases, such as user registration, property booking, content uploading, and error handling.

The rental platform is tested under different conditions, including low network connectivity, invalid user inputs, and simultaneous access from multiple users. This testing ensures:

- User actions like registration, search, and booking execute efficiently and accurately.
- The system handles errors such as failed uploads, invalid credentials, and slow internet connections.
- Property data updates in real time, ensuring accurate and consistent information for users and admins.
- The user interface remains intuitive, responsive, and consistent across devices.
- The system performs well during extended use, without crashes or performance degradation.

The following sections will outline the testing methodologies applied to ensure the platform's readiness for deployment, including integration testing, functional testing, performance testing, error handling, and security validation. By thoroughly testing the rental platform, we guarantee its robustness, scalability, and readiness for real-world use.

6.2 Integration Testing

Integration testing plays a crucial role in the system testing phase for the rental platform system, ensuring that all modules interact correctly and perform as expected. The platform consists of a React-based front-end, a Node.js back-end, and a MySQL database. It is essential to verify that all these components communicate seamlessly, especially in scenarios such as property listing, tenant and landlord interactions, bookings, and user authentication. The goal of integration testing is to detect and resolve issues related to data flow, API interactions, and module interoperability before deployment.

This phase ensures that:

- The React front-end correctly interacts with the Node.js API for actions like user login, property search, booking, and listing management.
- The Node.js back-end processes requests, performs business logic, and interacts accurately with the MySQL database.
- Property data, user profiles, and booking information are retrieved, updated, and stored without errors.
- The system handles user interactions and performs actions smoothly without disruptions or lag.

Integration Testing Approach

Integration testing in the rental platform system follows an incremental approach, where individual modules are first validated in isolation, followed by step-by-step integration. The key strategies applied are:

1. Bottom-Up Integration Testing

- Starts by testing low-level modules like user registration, property listing, and database queries.
- Ensures that the database (MySQL) and server-side logic (Node.js routes and models) work correctly before integrating them with the React UI.
- Confirms that data submitted via backend functions is stored and retrieved correctly.

2. Top-Down Integration Testing

- Focuses on testing high-level functionality from the user interface down to the database.
- Verifies how the front-end reacts to real-time inputs and API responses.
- Ensures that user interactions like searching properties or booking trigger accurate backend processes.

Key Integration Points and Testing Procedures

1. React Front-End and Node.js API Integration

- **Objective:** Validate that the React app sends accurate requests and correctly handles responses from the Node.js back-end.
- **Testing Steps:**
 - Simulate user login/signup processes and verify session/token handling.

- Perform CRUD operations on property listings, favorites, and user profiles.
 - Test the search functionality by entering various property criteria.
 - Handle invalid user inputs and simulate timeouts or server failures.
 - **Expected Outcome:**
 - UI updates dynamically with data from the API.
 - Users receive correct feedback (success, error messages).
 - API errors are gracefully handled without causing crashes.
2. **Node.js API and MySQL Database Integration**
- **Objective:** Ensure that the Node.js server interacts properly with the MySQL database to store and retrieve user data, property listings, and booking information.
 - **Testing Steps:**
 - Run insert, update, and delete operations for user and property data.
 - Verify database schema integrity and relations (e.g., user to property).
 - Simulate concurrent access by multiple users and test database performance under load.
 - Test database rollback on failed operations.
 - **Expected Outcome:**
 - All operations are reflected correctly in the database.
 - No data loss or inconsistency during high usage.
 - Secure handling of user credentials and session data.
3. **Node.js API and Property Booking System Integration**
- **Objective:** Ensure that the system properly handles property booking and availability updates.
 - **Testing Steps:**
 - Test booking functionality by simulating multiple users attempting to book the same property.
 - Verify that booked properties are correctly marked as unavailable in the system.
 - Simulate cancellations and verify that the property status updates accordingly.
 - **Expected Outcome:**
 - Booking information is correctly updated in the database.
 - The system prevents double-booking of properties.
 - Cancellation and status update workflows work seamlessly.
4. **Admin Panel and User Module Integration**
- **Objective:** Validate that user-submitted content (e.g., property listings, booking requests) appears in the admin panel and approved content is available to general users.
 - **Testing Steps:**
 - Submit a new property listing as a user.
 - Log in as an admin and review the approval workflows for property listings.
 - Check if approved properties appear on the platform's public pages.

- **Expected Outcome:**
 - Admin changes (approvals) are reflected instantly.
 - User-submitted properties go through the correct approval process.
 - Unauthorized content does not appear in public listings.

6.3 SYSTEM TESTING

6.3.1 Test Plan for SpotifyLite

Objective:

System testing aims to validate the end-to-end functionality of the rental platform system under real-world scenarios, ensuring the application meets all functional, performance, and usability requirements. This phase tests the entire system, including the user interface, API integration, database operations, and booking management, as a cohesive unit.

Scope of Testing:

- **Functional Testing:** Verifies that user features like registration, login, property search, listing creation, and booking functionality work correctly.
- **Performance Testing:** Measures app responsiveness, page load time, search speed, and booking process performance under different network conditions and heavy usage.
- **Usability Testing:** Ensures the user interface is intuitive, easy to navigate, and accessible across various devices and screen sizes, ensuring both tenants and landlords have an optimal experience.
- **Security Testing:** Tests the system's ability to protect user data, ensure secure user authentication, and prevent unauthorized access to personal and payment data.
- **Compatibility Testing:** Verifies app functionality on Android, iOS, and different browsers (for the admin panel).

Testing Environment

- **Frontend:** React.js, Tailwind CSS (Web)
- **Backend:** Node.js, Express.js (REST API)
- **Database:** MongoDB
- **Storage:** Cloud storage (e.g., AWS, Firebase, DigitalOcean)
- **Testing Tools:**
 - **Frontend Testing:** Jest, React Testing Library
 - **Backend Testing:** Postman, Supertest
 - **Performance Testing:** Lighthouse, Google PageSpeed Insights
 - **Database Testing:** MongoDB Compass, Robo 3T

6.3.2 System Test Cases

Test Case ID	Test Description	Input	Expected Result	Status
ST001	Verify user registration	Valid email, username, password	User account created successfully	✓
ST002	Verify login with valid credentials	Correct email and password	Redirected to Home page	✓
ST003	Verify login with invalid credentials	Incorrect email/password	Error message shown	✓
ST004	Search functionality	Search keyword "2BHK in Mumbai"	Properties matching the criteria displayed	✓
ST005	Add property listing	Property details (name, price, location)	Property appears in the landlord's dashboard	✓
ST006	Search for a property	Search keyword "3BHK near park"	Relevant properties displayed	✓
ST007	Handle no internet connection	Open app without network	"No internet connection" message shown	✓
ST008	Property availability after booking	Book a property and check availability	Property status updated to "Booked"	✓
ST009	View profile and edit details	Open Profile screen	User can edit and save profile info	✓
ST010	Logout functionality	Tap Logout button	Redirected to Login screen	✓
ST011	Verify booking process	Book a property (tenant)	Booking confirmation shown and booking added to dashboard	✓
ST012	Admin review and approval process	Admin reviews pending listings	Listing status updated to "Approved" for display	✓
ST013	Prevent duplicate registration	Same email used again	"Email already exists" message shown	✓
ST014	Unauthorized access check	Access admin URL without login	"Access denied" or redirected to login	✓
ST015	Multi-device session handling	Login on 2 devices	Last device overrides the previous session	✓

Pass/Fail Criteria

- **Pass:**
 - The feature performs its intended function correctly with valid inputs.
 - The system handles invalid inputs gracefully, displaying appropriate error messages.
 - User data, property listings, and booking details are displayed correctly and are updated in real-time.
 - The platform is responsive across all supported browsers.
- **Fail:**
 - Any deviation from the expected behavior, such as incorrect data rendering, failure to save user data, or errors in booking processes.
 - System crashes, particularly during critical operations like property listing, booking, or user authentication.
 - Inaccurate or missing data, especially in property listings, user profiles, or transaction history.
 - Any security lapse, such as unauthorized access or failure to encrypt sensitive data.

Risks and Mitigations

Risk	Impact	Mitigation
Slow property listing update	Delay in showing new property listings	Optimize database queries and use background tasks for asynchronous operations.
Booking conflicts	Multiple users trying to book the same property	Implement real-time booking status updates, and lock properties temporarily during booking.
API timeout	Broken data transactions	Use retry mechanisms, client-side timeout handling, and optimized API calls for faster responses.
Data loss during logout	User data not saved	Implement auto-sync for user data to ensure real-time updates before logout.
Cross-platform inconsistencies	UI breaks on iOS or web	Use responsive design (Tailwind CSS) and perform platform-specific testing to ensure consistency.

Security vulnerabilities	Unauthorized data access	Implement proper authentication, authorization, and encryption for sensitive data (e.g., passwords).
Property image upload failure	Poor user experience for landlords	Optimize image upload process, include file size limits, and provide feedback on upload status.
Data inconsistency in search	Users see incorrect or outdated listings	Ensure database consistency with transactional operations and implement caching mechanisms.
Scalability issues under load	Slow app performance during peak usage	Use load testing, optimize database queries, and consider scaling server infrastructure (e.g., cloud hosting).

SYSTEM MAINTENANCE

CHAPTER 7

SYSTEM MAINTENANCE

7.1 INTRODUCTION

The rental platform system is designed to offer essential features such as user authentication, property listings, booking management, and admin control over property uploads and user interactions. To ensure the application's long-term functionality, performance, and security, implementing a robust system maintenance strategy is essential.

Maintenance is an ongoing phase in the software development lifecycle that involves regular system monitoring, updates, bug fixes, and performance optimization. This phase ensures that the rental platform system adapts to evolving user needs, technological advancements, and security threats while maintaining a seamless user experience for both landlords and tenants. It also ensures the platform remains scalable, secure, and fully operational, providing users with accurate property listings, seamless booking processes, and smooth transactions.

The maintenance phase will focus on:

- Regular updates to the property listings, ensuring up-to-date and accurate information.
- Bug fixing and system optimizations to improve platform performance.
- Addressing user feedback and implementing feature enhancements to meet user needs.
- Implementing security patches to protect user data and ensure secure financial transactions.

With a well-structured maintenance approach, the rental platform will remain functional, secure, and user-friendly, helping it to grow and adapt in the dynamic market of real estate rental services.

7.2 SYSTEM MAINTENANCE

7.2.1 Importance of System Maintenance

1. Ensuring Seamless Property Listing and Booking

- **Continuous testing and optimization of property listings, booking system, and search functionalities are necessary to ensure a smooth user experience.**

- **Compatibility testing ensures that the platform runs efficiently across different devices and browsers, providing a seamless experience for both landlords and tenants.**

2. System Stability and Reliability

- **As the user base and property listings grow, components such as APIs, the database, and booking systems may encounter performance bottlenecks or slowdowns.**
- **Routine maintenance tasks such as code refactoring, database indexing, and API optimizations help to maintain the system's responsiveness, ensuring reliable service for users.**

3. Security and Data Protection

- **The platform handles sensitive user data, including personal details, property listings, and transaction information.**
- **Regular security updates to authentication mechanisms (e.g., encrypted passwords, token-based sessions) and backend APIs are critical to protect against vulnerabilities like SQL injection, session hijacking, and unauthorized access.**

4. Enhanced User Experience

- **Continuous feedback from users helps identify areas for improvement in the UI/UX design, ensuring smoother navigation, intuitive interfaces, and personalized user experiences.**
- **Routine bug fixes and performance optimization help to reduce system crashes and improve the platform's overall usability, making it more user-friendly for both landlords and tenants.**

5. Scalability and Feature Integration

- **As the user base grows or new features are introduced (e.g., payment processing, property reviews, or rental history tracking), the system must scale accordingly.**

- **Adaptive maintenance** ensures that new features or third-party integrations (e.g., payment gateways or external property data sources) are seamlessly incorporated into the platform without affecting the system's stability or performance.

7.2.2 Types of Maintenance Activities

1. **Preventive Maintenance** Preventive maintenance includes scheduled tasks that aim to prevent potential system failures or performance degradation, ensuring the rental platform remains reliable and efficient.

Key Activities:

- **Backend & Database Health Checks:** Periodically monitor API response times, query performance, and ensure proper database indexing for efficient data retrieval.
 - **Server and Hosting Optimization:** Regularly check and update hosting environments (e.g., cloud servers, storage services) to ensure they are operating efficiently and securely.
 - **Authentication and Session Handling Checks:** Validate secure login and token expiry mechanisms to prevent unauthorized access and ensure user data protection.
 - **Codebase Refactoring:** Conduct regular cleanup and documentation of the codebase to reduce complexity, fix potential bugs, and improve long-term maintainability.
2. **Corrective Maintenance** Corrective maintenance is triggered when bugs, crashes, or failures occur, and focuses on quickly identifying and fixing issues to restore normal functionality.

Key Activities:

- **Bug Fixing:** Address issues related to user authentication, property listing displays, booking failures, or user profile problems.

- **Server Downtime Recovery:** Resolve server-side crashes, performance issues, or service outages impacting the system's responsiveness or booking functionality.
 - **UI Fixes:** Correct layout problems, broken elements, or display errors that occur on different screen sizes or browsers.
 - **Data Recovery:** Restore lost or corrupted data, such as user profiles, booking details, or property listings, from backups to ensure data consistency.
- 3. Adaptive Maintenance** Adaptive maintenance ensures the system evolves to meet the changing requirements of users, technological advancements, or regulatory standards.

Key Activities:

- **Feature Additions:** Add new functionalities such as advanced search filters, rating and review systems, or new payment methods.
 - **Device Compatibility Updates:** Ensure the platform remains compatible with new operating system versions (Android/iOS) and various screen resolutions to deliver a consistent experience across devices.
 - **Third-party Service Integration:** Integrate external services such as payment gateways, property review platforms, or marketing tools for improved user experience and platform functionality.
 - **Regulatory Compliance:** Update privacy policies, terms of service, and features to comply with local laws or regulations (e.g., GDPR, data protection laws).
- 4. Perfective Maintenance** Perfective maintenance focuses on improving the existing functionalities of the platform to enhance performance, efficiency, and user satisfaction.

Key Activities:

- **Performance Tuning:** Optimize database queries, enhance page load speed, and reduce latency during property searches or booking processes.
- **UI/UX Improvements:** Continuously refine the user interface, improve navigation flow, and ensure a responsive design across all devices to provide a better user experience.
- **Caching and Lazy Loading:** Implement caching strategies and lazy loading techniques to reduce API calls, improve page load times, and decrease unnecessary resource usage.
- **Quality Enhancements:** Improve the quality of property listings, ensuring high-quality images, accurate descriptions, and seamless booking experiences based on user feedback.

These maintenance activities are critical for ensuring the rental platform remains efficient, reliable, and scalable over time, while also adapting to new user needs and technical environments.

Maintenance Strategy for SpotifyLite

A well-defined maintenance strategy is essential for ensuring the rental platform's long-term performance, security, and user satisfaction.

1. Continuous Monitoring and Logging

- Implement logging tools (e.g., Winston or Morgan for Node.js) to monitor backend requests, user actions, errors, and database interactions.
- Utilize analytics services (e.g., Google Analytics, Mixpanel) to track system load, booking success rates, and user behavior.
- Implement log aggregation systems (e.g., ELK Stack) for centralized error and performance tracking.

2. Scheduled Maintenance Routine

- Set up a maintenance calendar that includes periodic server cleanups, database optimizations, and third-party dependency updates.

- Assign clear roles for maintenance tasks (e.g., frontend maintenance, backend maintenance, and database updates) to ensure organized and efficient operations.

3. Automated Alerts and Error Detection

- Set up real-time alerts for critical issues, such as API failures, user login issues, database connectivity problems, or high server load using tools like **Firebase Crashlytics**, **Sentry**, or **New Relic**.
- Use monitoring tools (e.g., **Prometheus**, **Grafana**) to track system metrics and set up automated alerts for abnormal behavior like high CPU usage, memory usage spikes, or failed transactions.

4. Testing After Maintenance

- Perform regression testing after every update to ensure no previously functional features are broken. Use testing frameworks like **Jest** or **Mocha**.
- Test the application on multiple devices and browsers to ensure consistent behavior across platforms.
- Run performance tests (e.g., **LoadRunner**, **Apache JMeter**) after significant changes to measure the app's response time and stability under load.

Challenges in Maintaining SpotifyLite

Despite a comprehensive maintenance strategy, several challenges may arise during the platform's lifecycle:

1. Scalability and User Growth

- **Issue:** As the user base grows, the platform may experience performance degradation or server strain, leading to slower response times.
- **Solution:** Implement horizontal scaling with **load balancers** and **auto-scaling** infrastructure to distribute the load across multiple servers. Services like **AWS Elastic Beanstalk** or **Kubernetes** can manage scaling based on demand.

2. Network and Performance Issues

- **Issue:** Variations in user bandwidth can affect loading times, property browsing, and booking interactions, leading to poor user experience.
- **Solution:** Implement **lazy loading** for images and content, use **adaptive streaming** techniques, and allow offline access to key features using **local storage** or **service workers** for caching.

3. Security Threats

- **Issue:** Security threats such as **SQL injection**, **XSS (Cross-Site Scripting)** attacks, or **session hijacking** can compromise user data.
- **Solution:** Use **strong encryption** for sensitive data (e.g., hashing passwords with **bcrypt**), implement **token-based authentication** (e.g., **JWT**), and conduct **regular penetration testing** and security audits to detect vulnerabilities.

4. Dependency Failures

- **Issue:** Third-party services, libraries, or APIs may be deprecated or fail, disrupting system functionality.
- **Solution:** Keep dependencies up to date using tools like **npm** or **yarn** for JavaScript dependencies. Implement **fallback mechanisms** for critical external services (e.g., offer alternative payment gateways, or fallback to cached data for API calls).

SYSTEM SECURITY MEASURES

CHAPTER 8

SYSTEM SECURITY MEASURES

8.1 INTRODUCTION

This chapter outlines the **security measures** integrated into the **Rental Platform System**, a MERN-stack based property rental and listing web application, to ensure protection of user data, platform integrity, and overall system confidentiality. The platform facilitates interactions between landlords and tenants, making it essential to enforce strong security mechanisms across all components of the application.

Key security areas addressed include:

- **Application-level and device-level security**
- **Secure user authentication and authorization**
- **Encrypted data communication between client and server**
- **Database-level protections such as role-based access control (RBAC)**, encrypted storage, and regular secure backups

8.2 OPERATING LEVEL SECURITY

Ensuring robust operating-level security is essential for the **Rental Platform System** to deliver a secure, trustworthy experience for both landlords and tenants while safeguarding sensitive user and property data. At this level, several strategies are employed to enforce access restrictions, protect system resources, and maintain operational integrity across platforms.

Access control is managed using **strong authentication** mechanisms and **secure session handling**. Users are required to log in using verified credentials, and **token-based authentication** (e.g., JWT) ensures that only authorized users can access protected endpoints. **Role-based access control (RBAC)** restricts sensitive operations—such as editing property listings, verifying users, or managing user profiles—to specific roles like Admin, Landlord, or Tenant.

To protect local resources, the system employs **secure file handling** methods for managing profile pictures and property images. Storage mechanisms use access tokens and server-side validation to prevent unauthorized access to these resources.

Network security is reinforced using **firewalls and secure API endpoints**. Only authorized domains and verified users are allowed to communicate with the backend, reducing the risk of malicious traffic or injection attacks. HTTPS is enforced across all routes to encrypt data in transit.

A regular **update and patch management strategy** is followed to ensure all dependencies (Node.js, MongoDB drivers, React libraries, etc.) are up to date and vulnerabilities are patched quickly. This includes monitoring GitHub security alerts and using tools like npm audit.

Additionally, **runtime protection** strategies—such as monitoring API rate limits, validating user actions, and detecting anomalous behaviors—help protect the platform from brute-force attacks, abuse, or unauthorized data access.

Key Security Measures:

1. **Access Control & Authentication**
 - JWT-based secure login
 - Role-based access (Admin, Landlord, Tenant)
 - Secure session management
2. **File and Resource Protection**
 - Server-side validation for file uploads
 - Restricted access to sensitive resources (user images, property media)
 - Sanitization of user inputs
3. **Firewall & Network Security Rules**
 - HTTPS enforcement
 - CORS policy and origin whitelisting
 - Secure API keys and token validation
4. **Regular Updates & Patch Management**
 - Frequent updates to libraries and dependencies
 - Monitoring using npm audit and GitHub Dependabot alerts
5. **Intrusion Detection & Monitoring**
 - Suspicious login/activity alerts
 - API rate limiting and request validation
 - Monitoring server logs and user sessions

These practices ensure that the **Rental Platform System** remains secure, scalable, and resilient against operating-level threats, providing users with a reliable environment to manage property rentals effectively.

8.3 DATABASE LEVEL SECURITY

Database-level security is fundamental for the **Rental Platform System** to ensure that sensitive data—such as user credentials, property listings, personal details, and communication logs—are securely stored, accessed, and maintained. By enforcing strict security measures at the database layer, the system prevents unauthorized data access, manipulation, and breaches, while ensuring compliance and trustworthiness.

Authentication mechanisms serve as the first line of defense. The system uses **token-based authentication (JWT)** for secure client-server communication and can optionally integrate **multi-factor authentication (MFA)** for admin access to the backend dashboard. These practices help verify user identities and block unauthorized access to database resources.

Sensitive data, including passwords, user profiles, and landlord/tenant interactions, is encrypted both **at rest (e.g., hashed passwords using bcrypt)** and **in transit (via HTTPS and TLS)**. These layers of encryption ensure that data remains secure from interception and tampering.

The platform enforces **Role-Based Access Control (RBAC)** to manage permissions at a granular level. General users can only perform read and write operations on their own data, while admin roles are restricted to sensitive operations such as user verification, post moderation, and database-level audits.

To strengthen oversight, **database monitoring and auditing** tools are implemented to log activities such as login attempts, data retrievals, and changes. These logs are reviewed regularly to detect any suspicious behavior or unauthorized access attempts.

In case of service disruption, data loss, or attacks, the system follows a **backup and disaster recovery** protocol using automated backups and periodic data dumps. These ensure that the platform can quickly recover and resume operations without major data loss.

Key Security Measures:

1. Authentication Mechanisms

- Secure JWT tokens for user sessions
- Optional MFA for admin dashboards
- Passwords hashed using bcrypt

2. Data Encryption

- Encrypt sensitive data at rest (e.g., hashed credentials)
- Use SSL/TLS for encrypting data in transit

3. Role-Based Access Control (RBAC)

- Limit database operations based on roles (Admin, Landlord, Tenant)
- Prevent unauthorized CRUD operations

4. Database Monitoring & Auditing

- Log and review database access and modifications
- Detect anomalies and unauthorized access patterns

5. Backup & Disaster Recovery

- Automate daily/weekly database backups
- Implement rapid restoration procedures during downtime or attacks

8.4 SYSTEM LEVEL SECURITY

System-level security is paramount for SpotifyLite to protect user data, content, and interactions from unauthorized access and cyber threats. As SpotifyLite utilizes cloud-based services for streaming and user account management, securing the devices, communication channels, and updates is essential for maintaining the integrity and confidentiality of the system.

One of the primary security measures implemented is the use of secure boot mechanisms. This ensures that only verified and trusted software is loaded on the devices running SpotifyLite. During startup, the application checks for authenticated code to prevent malicious software from gaining control and tampering with the system.

For secure device authentication, SpotifyLite leverages robust authentication protocols like certificates, API keys, and cryptographic tokens. These mechanisms ensure that only authorized devices and applications can connect to the cloud backend, protecting the system from unauthorized access and potential data breaches.

The firmware update process is secured by using digitally signed and encrypted updates. This ensures that any updates to the mobile app, server-side software, or other components are verified before installation, preventing tampering with the app's functionality or the introduction of malicious code.

For secure communication, SpotifyLite uses TLS (Transport Layer Security) and HTTPS to encrypt data transmission between devices (such as mobile phones or web browsers) and the backend servers. These protocols ensure that sensitive information, including user preferences, playlists, and login credentials, are securely transmitted without risk of interception.

To monitor for threats, SpotifyLite implements continuous device monitoring and intrusion detection systems. These tools track unusual system activity, such as unauthorized access attempts or unexpected behavior, and alert administrators for prompt response and mitigation.

Key Security Measures:

1. **Secure Boot Mechanism** – Verifies that only trusted software runs on devices to prevent malicious code from compromising the system.
2. **Strong Authentication** – Utilizes certificates, API keys, or cryptographic tokens for secure device and service authentication.
3. **Secure Firmware Updates** – Ensures that app or server updates are digitally signed and encrypted to prevent unauthorized modifications.
4. **Encrypted Communication** – Employs TLS and HTTPS protocols to secure data transmission between devices and cloud services.
5. **Intrusion Detection & Monitoring** – Monitors devices and system activity to detect security breaches and proactively mitigate threats.

SYSTEM PLANNING AND SCHEDULING

CHAPTER 9

SYSTEM PLANNING AND SCHEDULING

9.1 INTRODUCTION

This chapter focuses on the **planning and scheduling** aspects involved in the development of the **Rental Platform System**, a full-stack MERN-based web application designed to connect property owners and potential tenants or buyers. The chapter outlines the strategic roadmap from initial concept to final deployment, emphasizing structured development and efficient resource utilization.

The planning phase includes identifying core features such as user registration, property listing creation, search and filter mechanisms, chat functionalities, and secure authentication. It also involves laying out the system architecture using the MERN stack—MongoDB, Express.js, React.js, and Node.js—ensuring each module is seamlessly integrated for optimal performance.

Scheduling involves breaking the development process into key milestones, such as frontend design, backend API setup, database modeling, and user testing. Resources such as developer time, hosting services, and third-party tools are allocated efficiently to avoid bottlenecks and delays. By adopting a well-organized project timeline with clear task distribution and deadline management, the development team ensures timely progress, reduced risk of errors, and improved collaboration. This approach ultimately contributes to delivering a user-friendly, secure, and scalable platform for renting and buying properties online.

9.2 PLANNING A SOFTWARE PROJECT

Planning the development of the **Rental Platform System** involves a systematic approach to ensure smooth execution and timely delivery. The first step is defining the **project scope**, which includes key features like user authentication, property listing, filtering, chat functionality, and profile management.

Next, **requirement gathering** is done by consulting stakeholders—developers, potential users, and mentors—to finalize the technical and functional needs. This helps in choosing tools like React, Node.js, MongoDB, and Express.

Resource allocation follows, assigning developers, tools (e.g., GitHub, Postman), and hosting platforms (e.g., Vercel/Render) to each module. A **Work Breakdown Structure (WBS)** is created to split the project into manageable tasks such as frontend design, backend API setup, and testing.

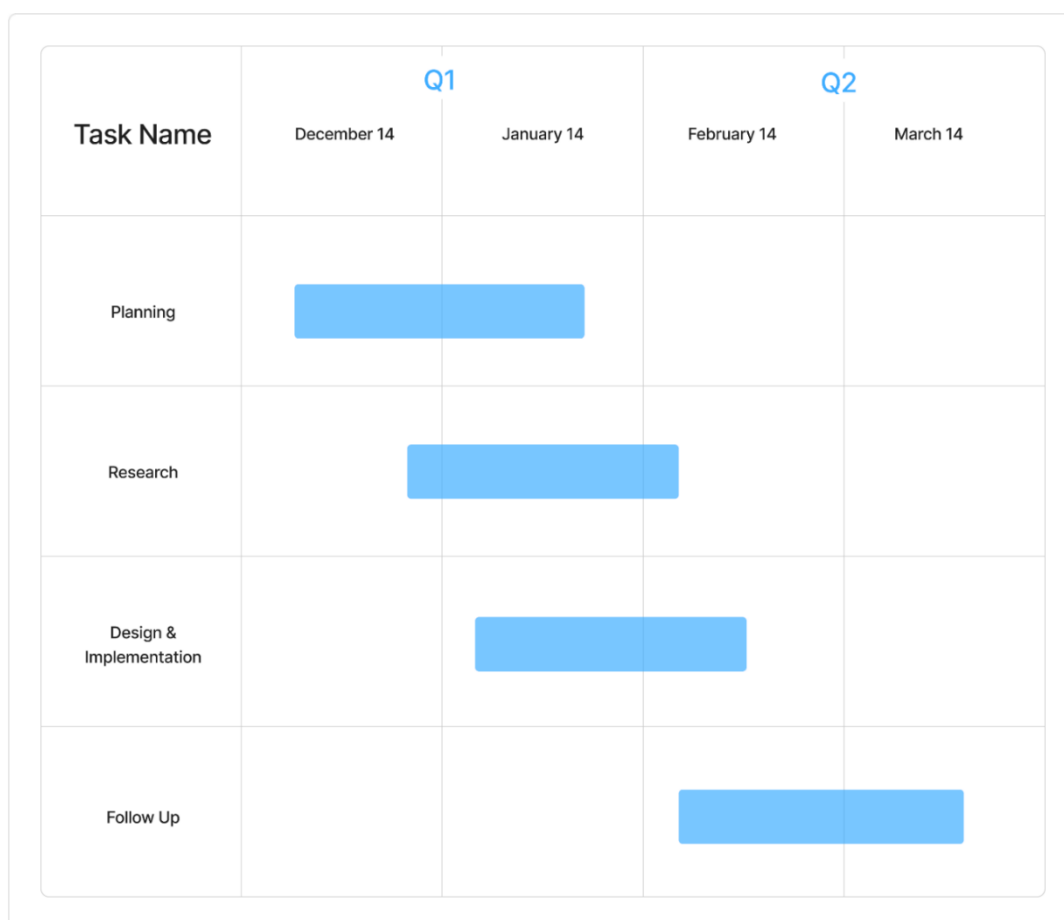
Finally, **time estimation** for each task is based on past experience and available working hours, ensuring a realistic and achievable development timeline.

Key Points:

1. **Define Scope** – Outline key features and goals.
2. **Gather Requirements** – Understand needs from users and team.
3. **Allocate Resources** – Assign tools, roles, and platforms.
4. **Break Down Tasks** – Create a clear WBS for execution.
5. **Estimate Time** – Set achievable deadlines for each task.

9.3 GANTT CHART

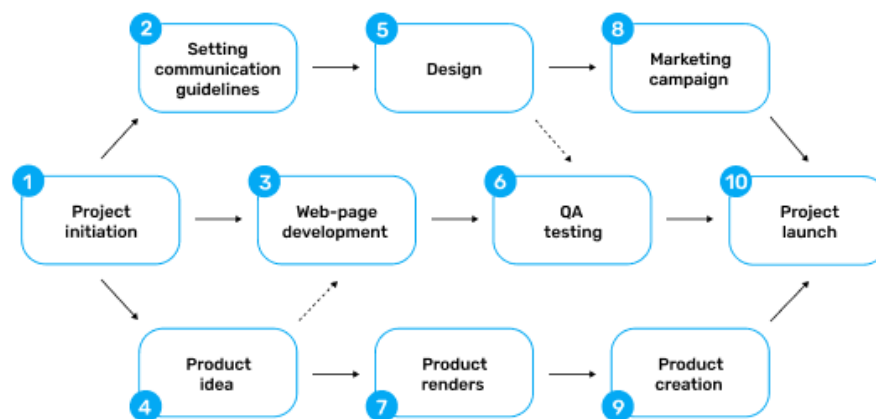
The Gantt Chart is a project management tool that helps plan, schedule, and track tasks for the SpotifyLite development project. It visualizes key activities, timelines, dependencies, and progress, ensuring tasks are completed on time and resources are efficiently allocated. This chart serves as a roadmap for successful project execution.



9.4 PERT CHART

A PERT chart is a project management tool that provides a graphical representation of a project's timeline. The Program Evaluation Review Technique (PERT) breaks down the individual tasks of a project for analysis.

PERT chart example



Tasks:

Dependencies:

Dependencies without resources:

SYSTEM COST ESTIMATION

CHAPTER 10

SYSTEM COST ESTIMATION

10.1 INTRODUCTION

This chapter outlines the estimated costs involved in developing, deploying, and maintaining the **Rental Platform System** built using the MERN stack. It considers key financial aspects such as development tools, hosting services, third-party integrations, and post-deployment support. By forecasting these expenses, we aim to ensure the project stays within budget while maintaining high-quality performance and scalability. This section presents a clear breakdown of the cost structure, enabling better financial planning and resource optimization throughout the project's lifecycle.

10.1 LOC BASED ESTIMATION

LOC-Based Estimation is a software cost estimation technique that evaluates the overall project cost based on the number of Lines of Code (LOC) required. For the **Rental Platform System**, this method helps estimate the development effort, time, and budget by analyzing the amount of coding work involved across all modules such as property listings, user authentication, chat functionality, and profile management.

The estimation begins by approximating the total LOC needed for the entire project. In the case of a MERN stack application, this includes JavaScript code for the frontend (React), backend (Node.js and Express), and MongoDB queries. The cost per LOC depends on factors like feature complexity, team expertise, and tools used.

Once the total LOC is estimated, it is multiplied by the average cost per line of code to derive the project's cost. Though simple, this technique provides a useful baseline and can be refined using complementary approaches such as Function Point Analysis or reference to similar past projects.

10.2 FUNCTION POINT BASED ESTIMATION

Function Point-Based Estimation (FPA) is a software estimation technique that focuses on the functionality and complexity of a system rather than its lines of code. For the **Rental Platform System**, this method helps evaluate the effort required by analyzing core features such as property listings, user registration/login, search and filter options, messaging between users, and profile management.

In FPA, functional components like inputs (e.g., property form submissions), outputs (e.g., search results), user interactions (e.g., login, chat), and external interfaces (e.g., API calls

to the backend) are identified and assigned complexity-based weights. These are then used to calculate the total number of function points (FPs).

The total FPs are mapped to effort using historical productivity data, which indicates how many FPs a developer can handle within a certain time frame. FPA also takes into account non-functional aspects like performance, security, and scalability—important for a platform where many users may search or post listings simultaneously.

This estimation method is ideal for the **Rental Platform System**, as it offers a technology-independent way to plan and manage project timelines and resources based on the system's functionality rather than the code volume.

FUTURE ENHANCEMENT

CHAPTER 11

FUTURE ENHANCEMENT

11.1 INTRODUCTION

The **Rental Platform System** is a user-friendly web application designed to connect property owners with potential tenants and buyers. Developed using the MERN stack (MongoDB, Express.js, React, Node.js), the platform allows users to list, search, and manage rental or sale properties with ease. It includes essential features such as user authentication, property listing creation, search and filter functionalities, profile management, and a built-in messaging system for communication between users. Optimized for performance and scalability, the platform provides a seamless experience for users across different devices. By simplifying the property rental and buying process, the system addresses the needs of both landlords and tenants in a centralized, efficient, and accessible way.

11.2 MERITS OF THE SYSTEM

One of the key advantages of the **Rental Platform System** is its efficiency in connecting property owners and potential tenants or buyers, offering a seamless and user-friendly interface. The platform allows easy property listing creation, search, and communication, ensuring that users can quickly find suitable properties without hassle.

Another significant merit is the scalability and performance optimization of the system. Built using the MERN stack, the platform ensures smooth operation even as the number of users and listings grows, offering quick response times and handling high traffic loads effectively.

The system's search and filter functionalities provide users with a highly customizable experience, allowing them to find properties based on various criteria such as location, price range, and property type. This increases user satisfaction and enhances the overall usability of the platform.

Additionally, the messaging system integrated into the platform allows landlords and tenants to communicate directly within the app, facilitating seamless interaction and quick resolution of inquiries or concerns.

Finally, the **Rental Platform System** offers an accessible and efficient solution for property rental and purchase, making it an attractive option for both landlords and tenants looking for a reliable and modern way to manage property listings.

11.3 LIMITATIONS OF THE SYSTEM

While the **Rental Platform System** offers numerous benefits, there are certain limitations that could impact the user experience. One major limitation is the absence of integrated payment processing within the platform. Although users can view and contact property owners for rental or purchase inquiries, they cannot complete transactions directly through the system, which may reduce convenience for some users who prefer an all-in-one solution.

Another limitation is the dependency on user-generated content for property listings. Since landlords must manually upload property details and photos, there may be inconsistencies in the quality and accuracy of listings, potentially leading to a suboptimal user experience. Moreover, without the verification of listings, there could be issues related to misleading or inaccurate property information.

Additionally, the system's search functionality, while effective, might not be as advanced as some larger platforms, limiting the precision of property search results. Users may experience difficulty finding specific properties based on more nuanced criteria, which could lead to a less efficient property search process.

Finally, as the platform is currently built with a focus on core features, advanced options such as property pricing analytics or real-time market insights may not be included. Users seeking in-depth data or specialized tools for property investment or market analysis may find the platform lacking in these areas.

11.4 FUTURE ENHANCEMENTS OF THE SYSTEM

To improve the user experience and functionality of the **Rental Platform System**, several future enhancements can be considered. One potential improvement is the integration of a payment gateway, enabling users to complete rental or purchase transactions directly through the platform. This would streamline the process and provide a more convenient and secure way for users to finalize deals with landlords, making the platform more competitive and user-friendly.

Another key enhancement could be the addition of advanced property filtering and search options. By incorporating more detailed search criteria, such as property amenities, square footage, or specific location preferences (e.g., proximity to public transport), users could find their ideal rental or purchase more efficiently. Additionally, incorporating AI-driven recommendations based on user behavior and preferences could further personalize the property search experience.

Integrating a property verification system could also be a significant improvement. Implementing a feature where landlords can verify their listings through a document submission or authentication process could increase trust and reliability in the platform, reducing the risk of fraudulent or misleading property information.

Lastly, expanding the platform to include features such as property reviews and ratings could help foster a more interactive community. Allowing tenants to leave reviews on properties or landlords could create a more transparent environment and assist prospective tenants in making informed decisions. This feature could also help improve landlord accountability and attract more users to the platform.

CONCLUSION

CHAPTER 12

CONCLUSION

The **Rental Platform System** aims to connect landlords and tenants for both rental and property purchase transactions. The platform is designed to streamline the property rental process, offering users a user-friendly interface to search, filter, and manage rental properties. With an intuitive design, efficient property listing management, and secure communication features, the system provides a comprehensive solution for users seeking to rent or buy properties. By ensuring smooth transactions and maintaining transparency, the Rental Platform System can cater to a wide audience, especially those in urban areas and emerging markets.

System Merits

One of the key advantages of the **Rental Platform System** is its ability to connect users with landlords in an efficient and transparent manner. The platform offers features such as property search and filtering, detailed property listings, and a secure messaging system for both parties to communicate. With these features, users can easily find suitable properties while landlords can reach a larger pool of potential tenants or buyers. Additionally, the platform's user-friendly design makes it easy for users of varying tech-savviness to navigate and find properties that match their preferences.

Another merit is the centralized management of property listings and user data, which ensures that both landlords and tenants can easily manage and track their listings, applications, and communication. Furthermore, the system helps build trust by including features like reviews and ratings for landlords, which adds credibility to the properties listed.

System Limitations

Despite its advantages, the **Rental Platform System** does have some limitations. One challenge is the absence of an integrated payment gateway, which could hinder the completion of transactions directly through the platform. Users must look for external payment methods, which can complicate the process. Additionally, while the platform provides basic property filtering and search functionalities, there is room for improvement in terms of advanced search options and personalized recommendations based on user preferences.

Another limitation is the lack of property verification, which could create trust issues. Although reviews can help alleviate this to some extent, users may still be hesitant to rely on listings without formal verification. The absence of detailed information on property amenities or neighborhood data can also lead to users making uninformed decisions.

Lastly, while the platform is designed for use on a wide range of devices, performance can vary, especially on older smartphones or low-end devices. Users with limited resources may face challenges navigating through multiple property listings or using advanced features.

Future Enhancements

To further improve the **Rental Platform System** and meet user demands, several enhancements can be considered:

1. **Payment Gateway Integration:** Adding a payment gateway would simplify transactions, allowing users to complete rental or property purchase deals directly through the platform. This would increase convenience and security for all users.
2. **Enhanced Search and Filters:** Expanding the search filters to include more granular options such as property amenities, square footage, or proximity to key landmarks could improve the property search experience. Additionally, incorporating AI-powered search recommendations based on user behavior and preferences could make the platform more intuitive.
3. **Property Verification System:** Introducing a formal property verification system would help ensure the legitimacy of listings and build trust among users. This could involve document verification or third-party inspections.
4. **Review and Rating System Expansion:** Expanding the review system to include both landlords and tenants could further promote transparency and trust, allowing users to make more informed decisions.
5. **Offline Functionality:** Incorporating offline features, such as the ability to save property listings for offline viewing, would allow users to access important information even in areas with limited connectivity.
6. **Mobile App Development:** While the platform is web-based, developing a dedicated mobile application for both iOS and Android would enhance the user experience, especially for users who prefer browsing and interacting through mobile devices.
7. **AI-Powered Recommendations:** Leveraging AI to provide personalized property suggestions based on user preferences, location, and past search behavior would enhance the search experience and keep users engaged.
8. **Social Features:** Introducing features like property sharing, allowing users to share listings with friends or family, could increase platform engagement and enhance word-of-mouth marketing.
9. **Integration with Smart Devices:** Integrating with smart home devices could provide additional features, such as virtual tours or real-time notifications for new properties.
10. **Collaboration with Real Estate Agencies:** Partnering with real estate agencies to ensure access to a wide range of properties could help boost the platform's inventory and appeal to users seeking professional assistance.

BIBLIOGRAPHY

CHAPTER 13

BIBLIOGRAPHY

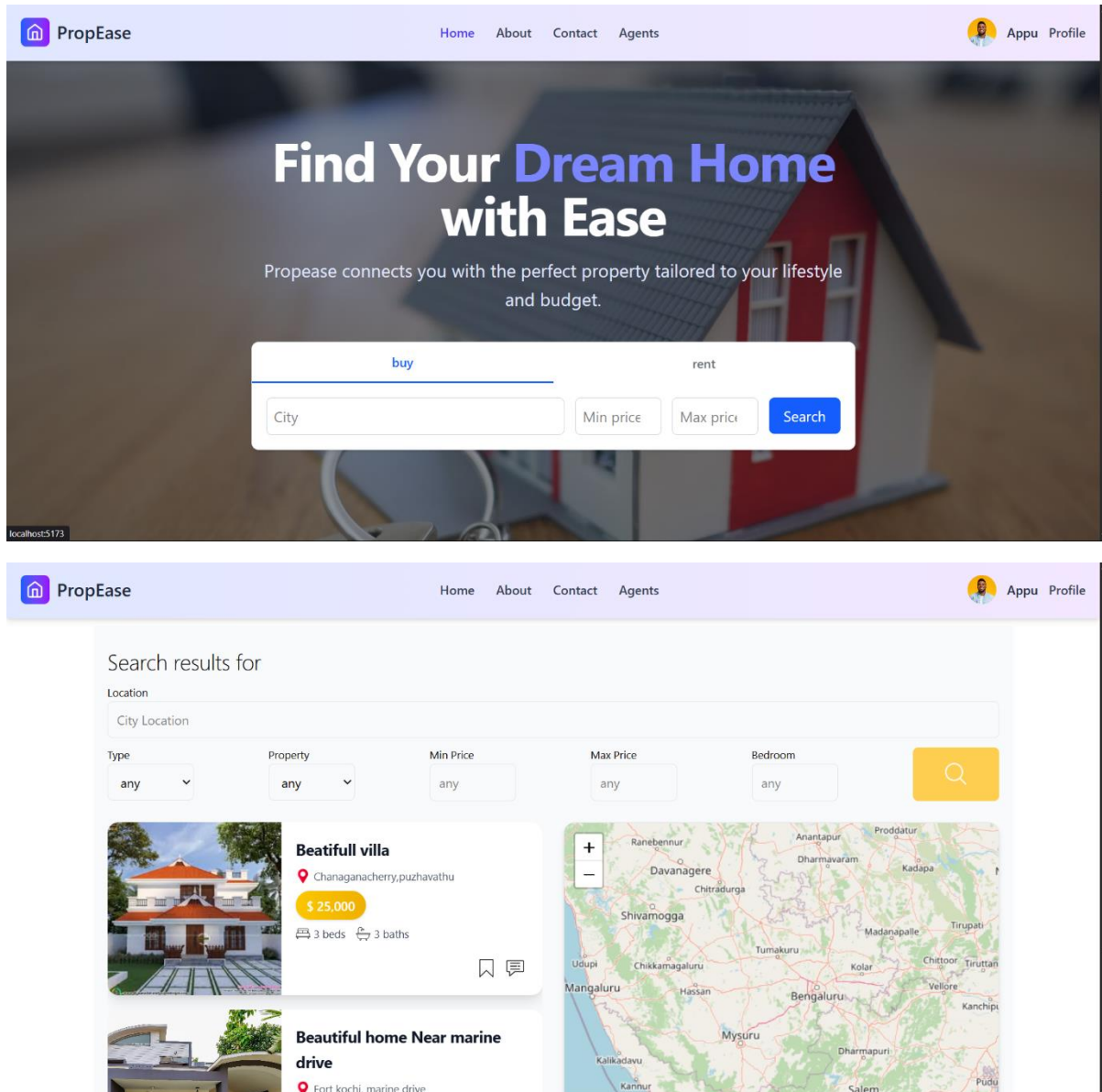
1. **W3Schools.** (2023). HTML and CSS Tutorial. Retrieved from <https://www.w3schools.com>
2. **MDN Web Docs.** (2023). JavaScript Documentation. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
3. **MongoDB.** (2023). MongoDB Documentation. Retrieved from <https://www.mongodb.com/docs>
4. **Express.js.** (2023). Express.js Framework Guide. Retrieved from <https://expressjs.com>
5. **ReactJS.** (2023). React Documentation. Retrieved from <https://reactjs.org/docs/getting-started.html>
6. **Node.js.** (2023). Node.js Documentation. Retrieved from <https://nodejs.org/en/docs/>
7. **GitHub.** (2023). GitHub Repositories for MERN Stack Projects. Retrieved from <https://github.com>
8. **Stack Overflow.** (2023). MERN Stack Development and Troubleshooting. Retrieved from <https://stackoverflow.com>
9. **Dev.to.** (2023). MERN Stack Articles and Tutorials. Retrieved from <https://dev.to>
10. **Statista.** (2023). Real Estate Market Insights and Property Trends. Retrieved from <https://www.statista.com>

APPENDIX

CHAPTER 14

APPENDIX

SCREENSHOT



 PropEase

Home About Contact Agents

 Appu Profile



Beatifull villa

Chanaganacherry,puzhavathu, Chanaganacherry

rent house

Property Features

General

Utilities: Tenant pays for all utilities
Parking policy: Parkings available
Advance: Advance is Required

Room Size

Bedrooms	Bathrooms	Size
3	3	2000 sqft

Nearby Places

Schools: 3

 PropEase

Home About Contact Agents

 Appu Profile



Appu

appu@gmail.com

Update Profile Logout

My List

+ Add New Post



Beatifull villa

Chanaganacherry,puzhavathu

\$ 25,000

3 beds 3 baths

Messages

Messages



blitz... 06:44 PM
No messag...



blitzbugg

 PropEase

Home About Contact Agents

Sign InSign Up

Welcome back

Login

Don't you have an account?



 PropEase

Home About Contact Agents

Sign InSign Up

Create an Account

Register

Do you have an account?



 PropEase

HomeAboutContactAgents

 Appu Profile

Title

Price

Address

Description

B / U H1 H2 • List 1. List Quote Divider Image Undo

Redo

No images uploaded yet. Upload some images to showcase your property.

Upload

 PropEase

HomeAboutContactAgents

 Appu Profile

Username

Appu

Email

appu@gmail.com

Password

Update



Upload